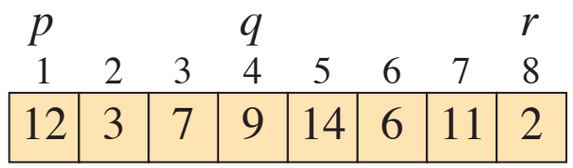
Algorithm Analysis and Elementary Sorting Algorithms

Lecture II for course in Data Structures & Algorithms for AI

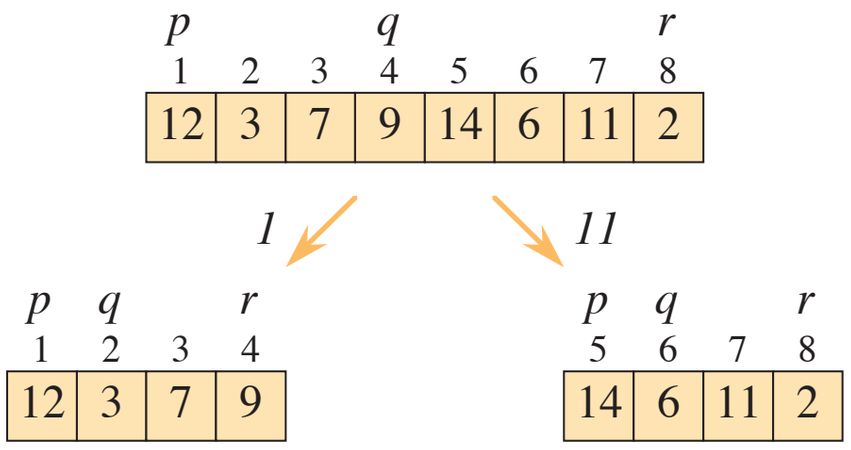
Lectures by Hannah Santa Cruz Baur. Based on *Introduction to Algorithms*, Cormen et al.

Merge sort



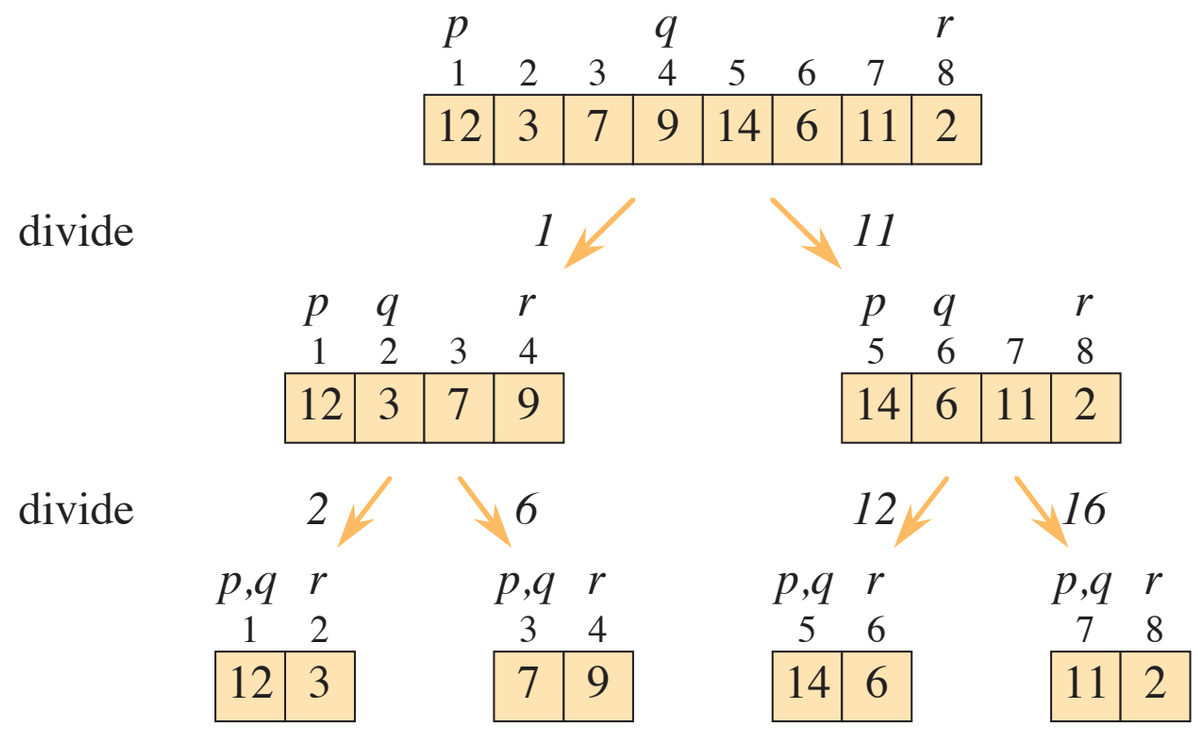
Merge sort

divide



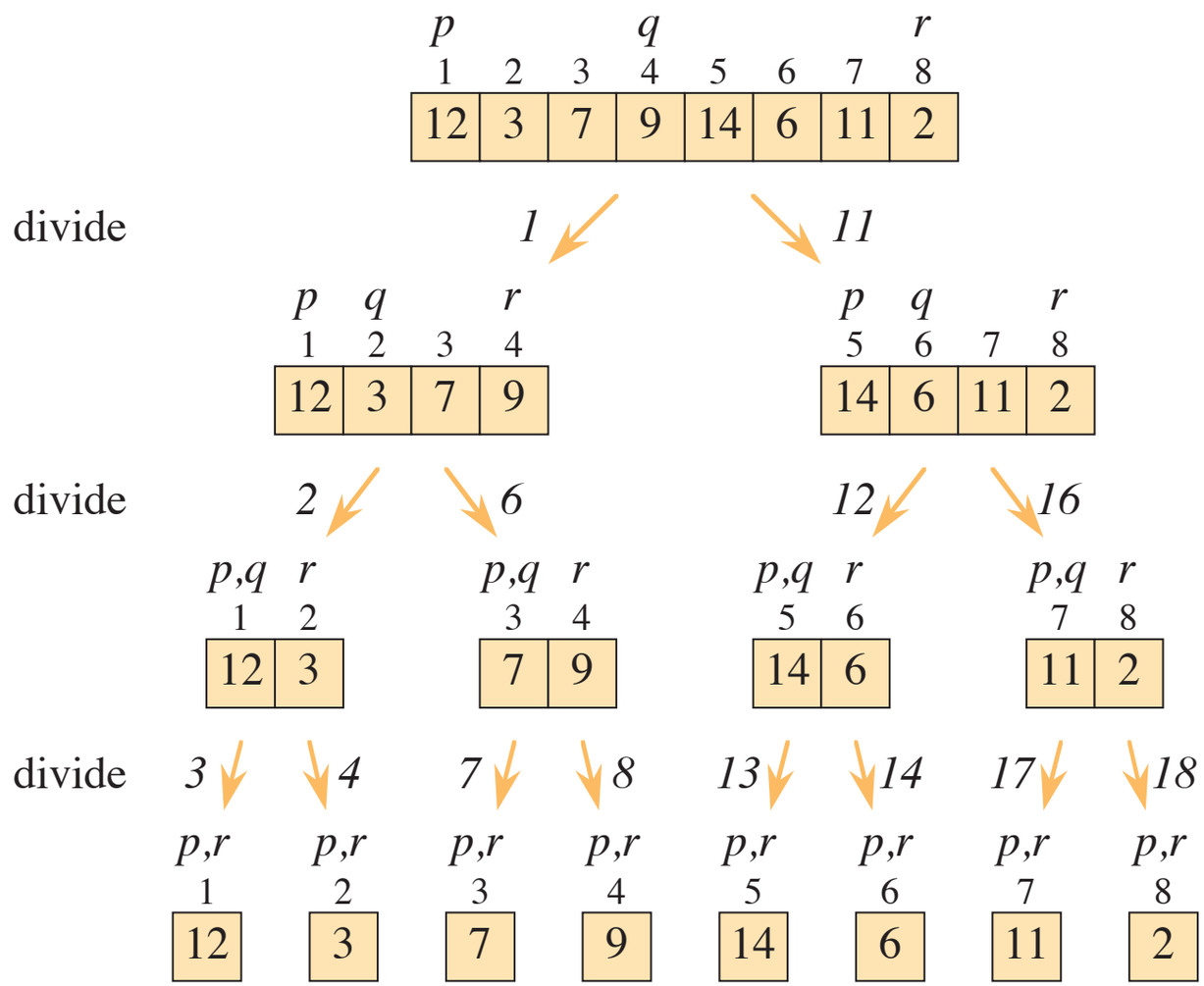
- splits an array into two subarrays down the middle,

Merge sort



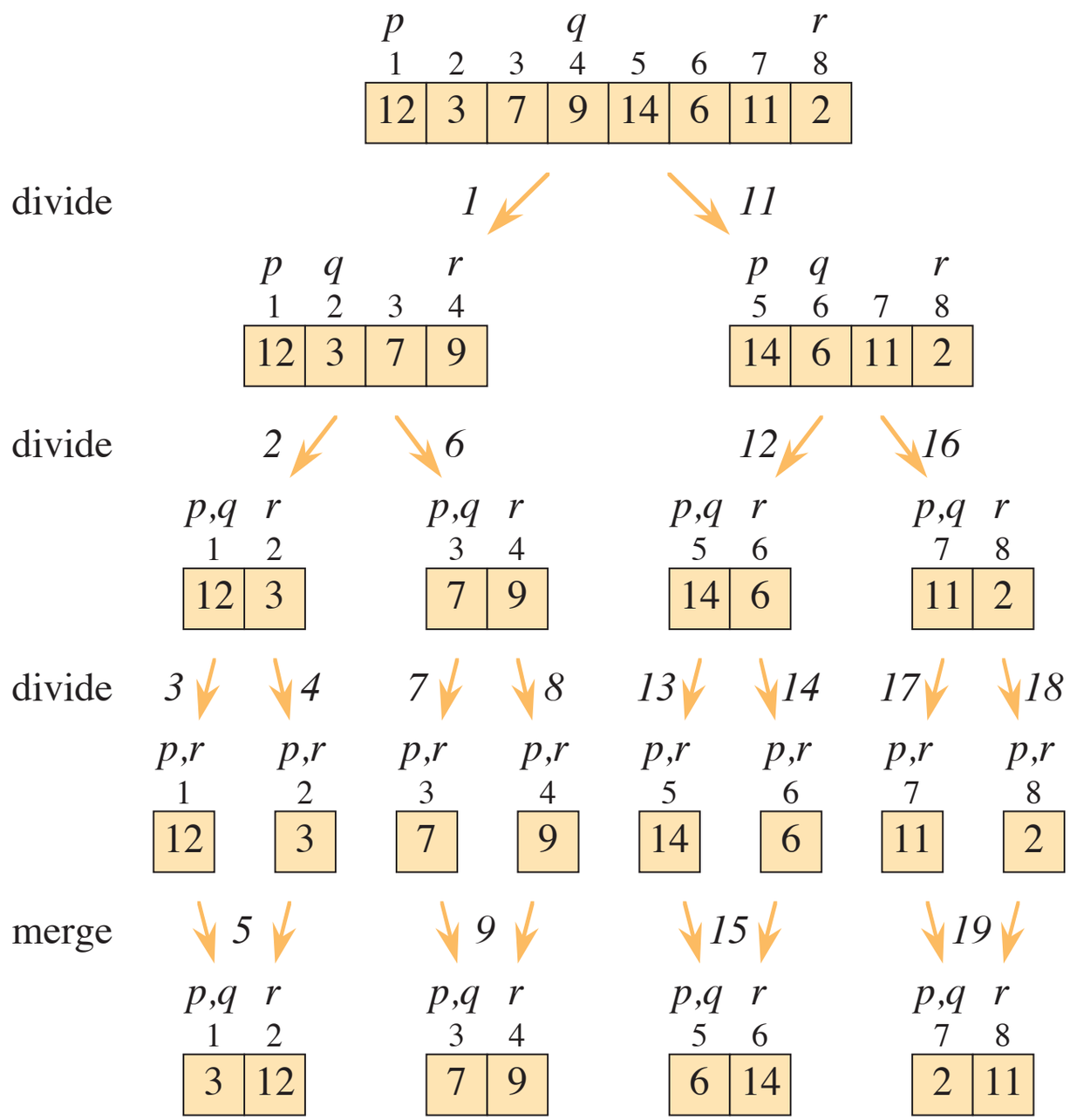
- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,

Merge sort



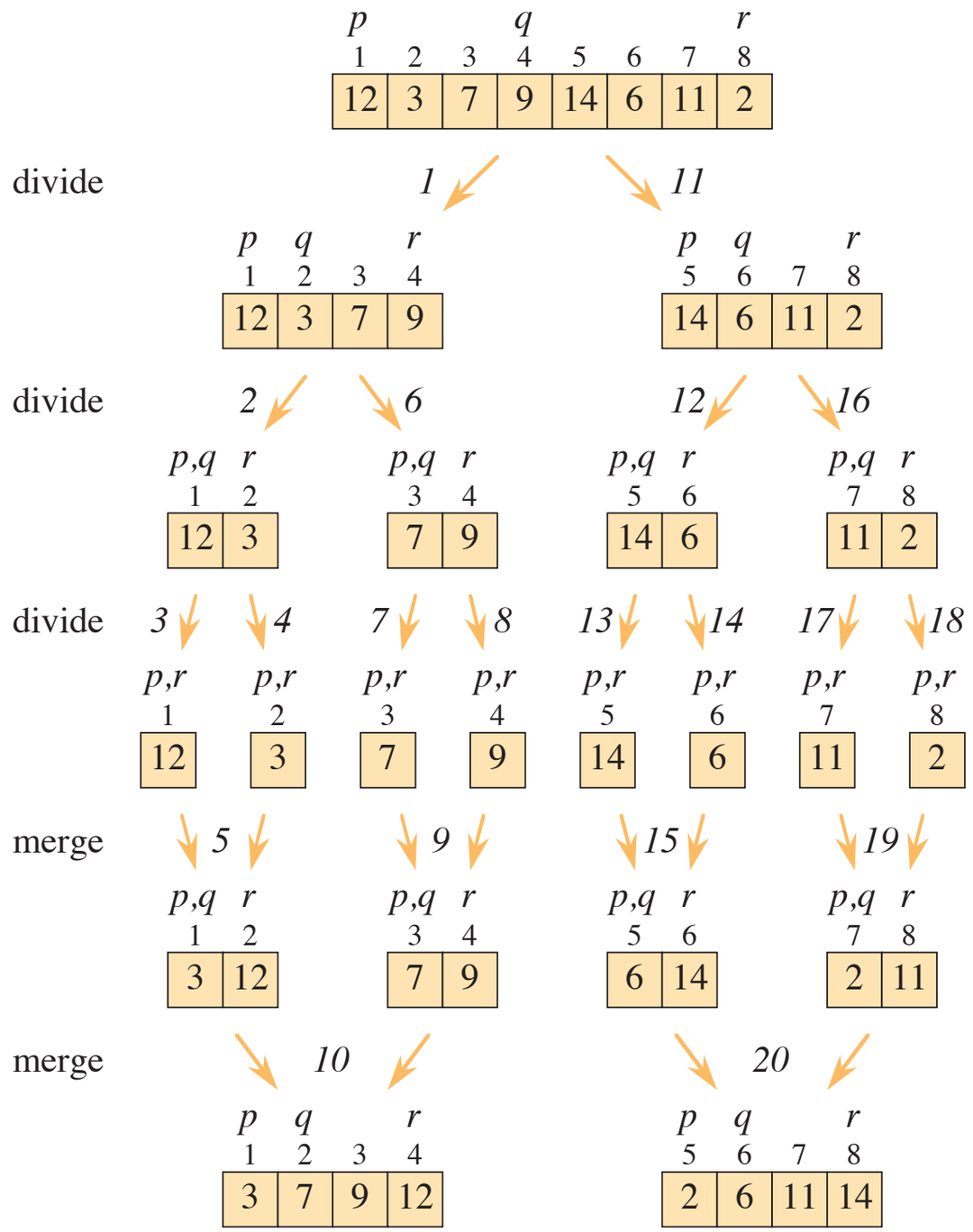
- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,

Merge sort



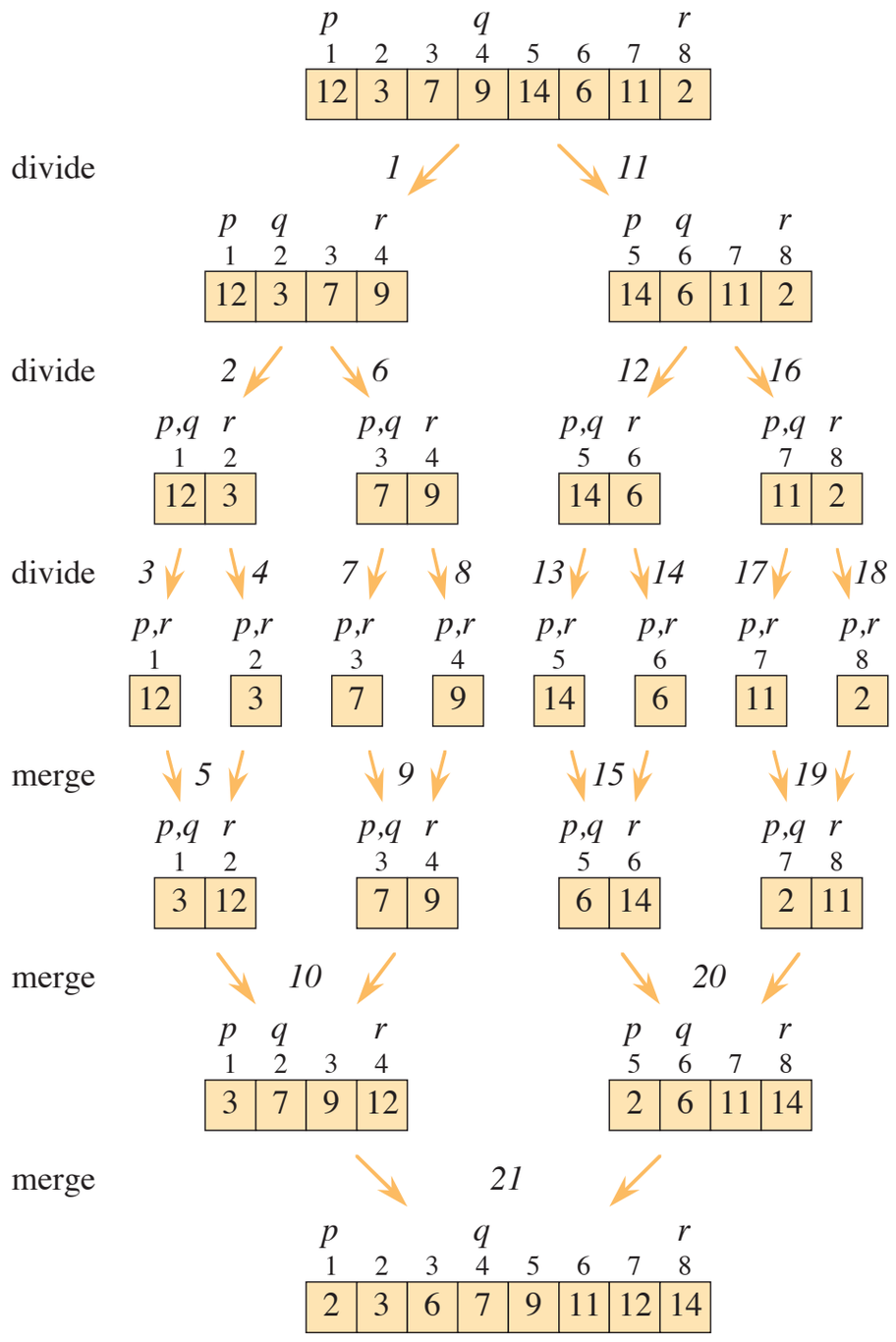
- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,
- and finally combines the adjacent sorted subarrays

Merge sort



- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,
- and finally combines the adjacent sorted subarrays

Merge sort



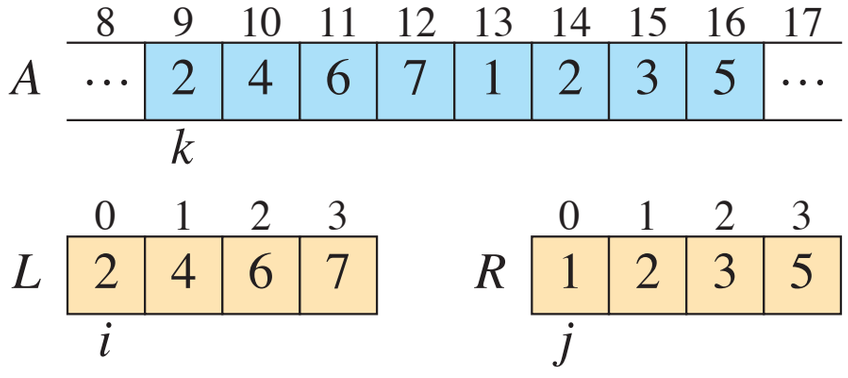
- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,
- and finally combines the adjacent sorted subarrays, into one singular sorted array.

Merge sort

- Consider an array A, with entries at indexes $p, \ldots, q$ to be sorted

```
MERGE-SORT(A, p, r)
1   if p ≥ r                                  // zero or one element?
2       return
3   q = ⌊(p + r)/2⌋                           // midpoint of A[p : r]
4   MERGE-SORT(A, p, q)                       // recursively sort A[p : q]
5   MERGE-SORT(A, q + 1, r)                   // recursively sort A[q + 1 : r]
6   // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7   MERGE(A, p, q, r)
```

The MERGE procedure



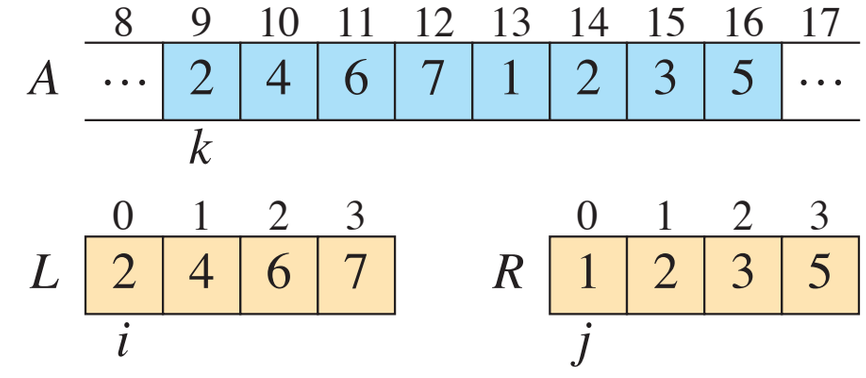
Consider the concatenated array A of two sorted arrays L and R.

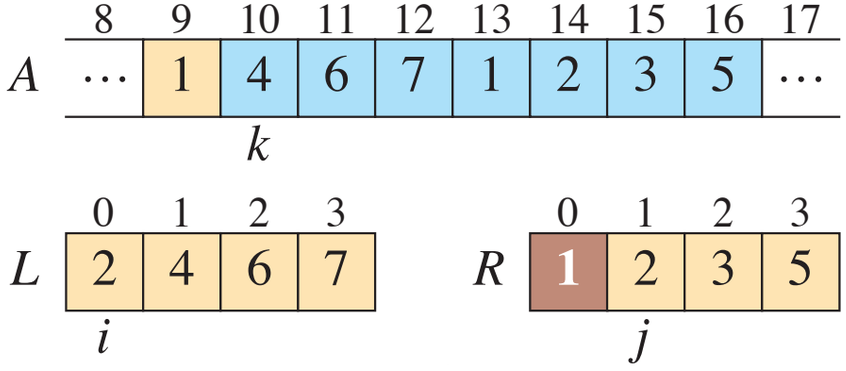
We wish to fully sort the array, by utilizing the fact that L & R are already sorted.

To save storage, we will overwrite the values in A.

We indicate with blue the elements of A that have yet to be overwritten.

The MERGE procedure





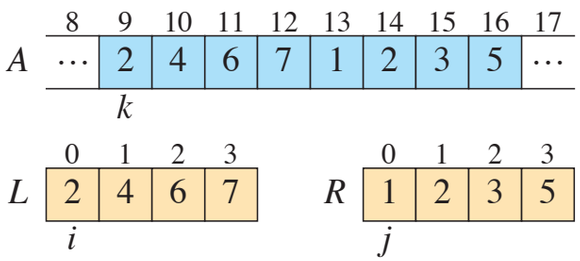
We overwrite the first element of A with the smallest element of L & R.

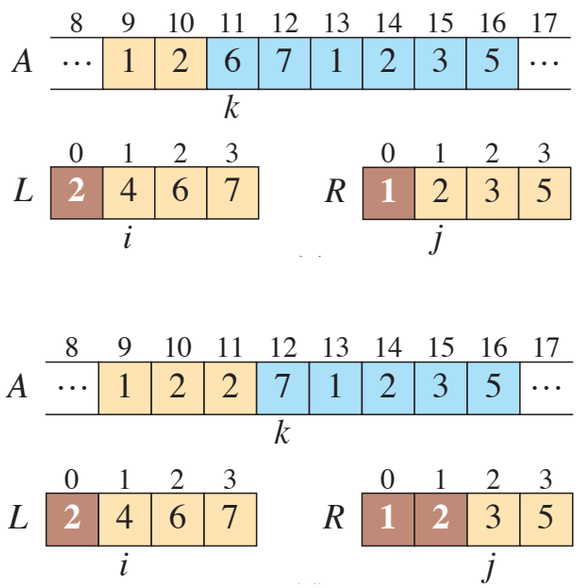
Because L & R are sorted, it suffices to compare their first elements to find the smallest.

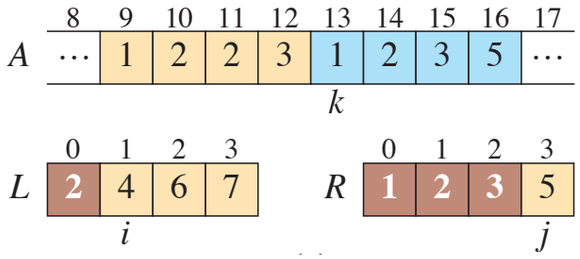
The first element of A is now tan, to indicate it is the final element in its position.

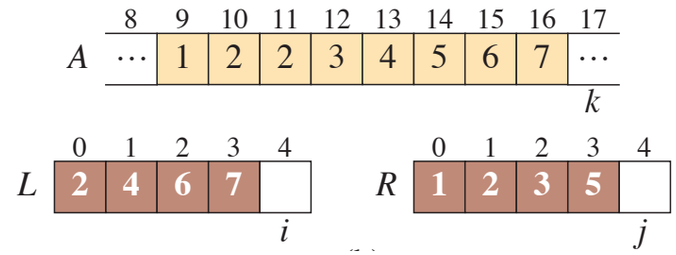
The first element of R is now brown, to indicate it has been copied over.

The MERGE procedure









We continue so, until all elements of A have been overwritten.

This outputs a sorted array A, corresponding to the merger of sorted arrays L & R.

Analysis of merge sort

- We compute the worst case running time of merge sort, $T(n)$, recursively, distinguishing the three stages of the algorithm:
  - **Divide** computing the middle of the array takes constant time $\Theta(1)$.
  - **Conquer** recursively solving two subproblems, each of size $n/2$, takes a total running time of $2T(n/2)$.
  - **Combine** The merge procedure takes $\Theta(n)$ time.
- Adding them together, we get

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(nlog(n))$$

Quick Sort
and
Bucket Sort

Lecture IV for course in
Data Structures & Algorithms for AI

Lectures by Hannah Santa Cruz Baur. Based on *Introduction to Algorithms*, Cormen et al.

Divide and conquer algorithms

- Another popular divide and conquer algorithm is Quick sort:
  - **Divide:** partition into two subarrays- such that all elements in one subarray are smaller than a chosen **pivot**, and those in the other subarray are in larger than this pivot.

Divide and conquer algorithms

- Another popular divide and conquer algorithm is Quick sort:
  - **Divide:** partition into two subarrays- such that all elements in one subarray are smaller than a chosen **pivot**, and those in the other subarray are in larger than this pivot.
  - **Conquer:** recursively calls itself to sort the smaller arrays.

Divide and conquer algorithms

- Another popular divide and conquer algorithm is Quick sort:
  - **Divide:** partition into two subarrays- such that all elements in one subarray are smaller than a chosen **pivot**, and those in the other subarray are in larger than this pivot.
  - **Conquer:** recursively calls itself to sort the smaller arrays.
  - **Combine:** join the two arrays, the union is immediately sorted, there is no need for any more steps.

Divide and conquer algorithms

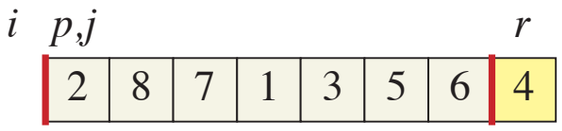
- Another popular divide and conquer algorithm is Quick sort:
  - **Divide:** partition into two subarrays- such that all elements in one subarray are smaller than a chosen **pivot**, and those in the other subarray are in larger than this pivot.
  - **Conquer:** recursively calls itself to sort the smaller arrays.
  - **Combine:** join the two arrays, the union is immediately sorted, there is no need for any more steps.
- Compare with **merge sort**, which:
  - splits an array into two subarrays down the middle,
  - recursively calls itself to sort the smaller arrays,
  - combines the sorted subarrays, by using the MERGE procedure to create one sorted array.

QUICK SORT

- Consider an array A, with entries at indexes $p, \ldots, r$ to be sorted

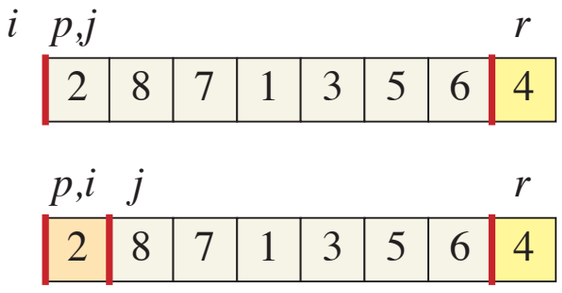
```
QUICKSORT(A, p, r)
1   if p < r
2       // Partition the subarray around the pivot, which ends up in A[q].
3       q = PARTITION(A, p, r)
4       QUICKSORT(A, p, q − 1)  // recursively sort the low side
5       QUICKSORT(A, q + 1, r)  // recursively sort the high side
```

The PARTITION procedure



The entry at index r will be the pivot.

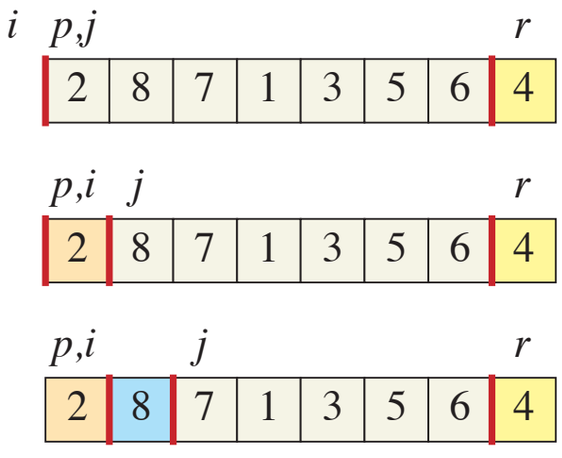
The PARTITION procedure



The entry at index r will be the pivot.

Tan elements have been added to the "low subarray" - those entries with value smaller than the pivot.

The PARTITION procedure

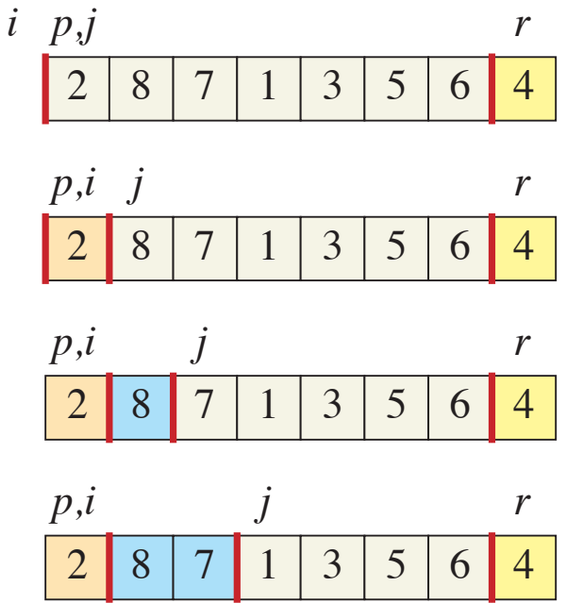


The entry at index r will be the pivot.

Tan elements have been added to the "low subarray" - those entries with value smaller than the pivot.

Blue elements have been added to the "high subarray"

The PARTITION procedure



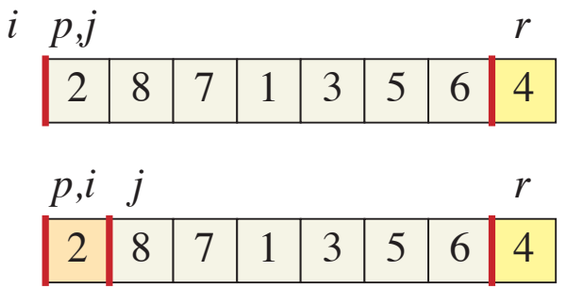
The entry at index r will be the pivot.

Tan elements have been added to the "low subarray" - those entries with value smaller than the pivot.

Blue elements have been added to the "high subarray"

Elements are combed from left to right, white elements are yet to be classified.

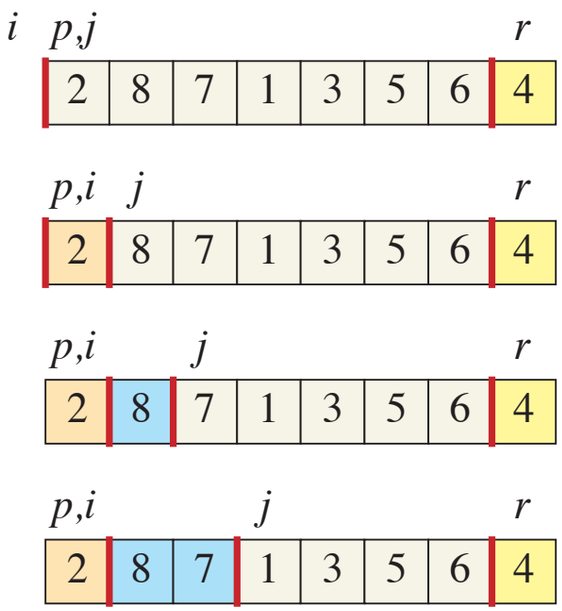
The PARTITION procedure



The entry at index r will be the pivot.

Value *2* is swapped with itself, being placed in the *low subarray*.

The PARTITION procedure

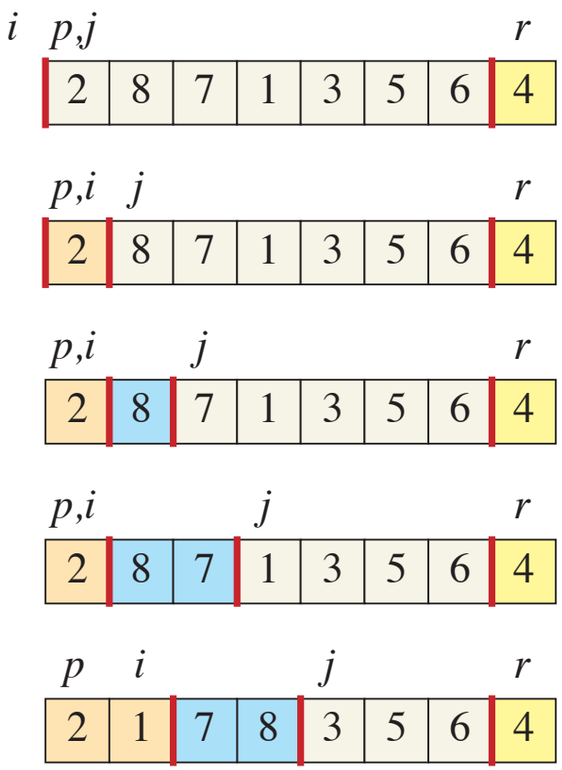


The entry at index r will be the pivot.

Value *2* is swapped with itself, being placed in the *low subarray*.

Values *8, 7* are swapped with themselves, being placed in the *high subarray*.

The PARTITION procedure



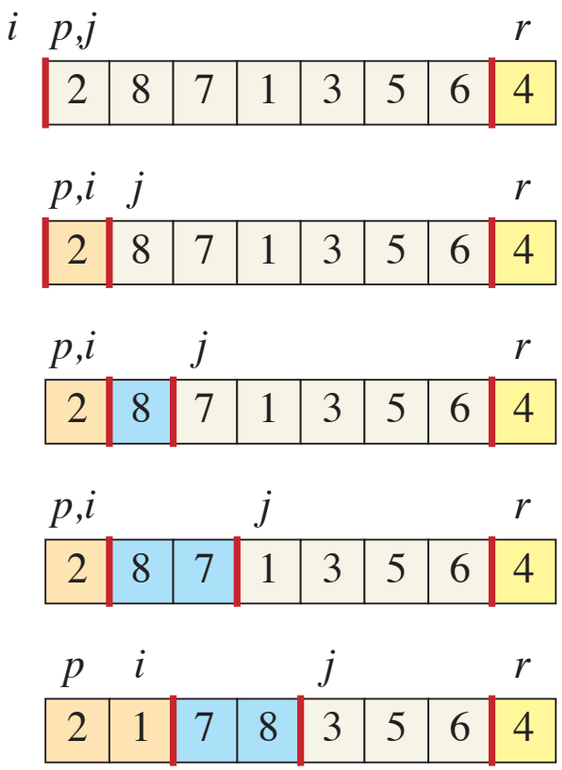
The entry at index *r* will be the pivot.

Value *2* is swapped with itself, being placed in the *low subarray*.

Values *8, 7* are swapped with themselves, being placed in the *high subarray*.

Value *1* is swapped with value *8*, placed into the *low* & *high subarrays* respectively.

The PARTITION procedure



The entry at index r will be the pivot.

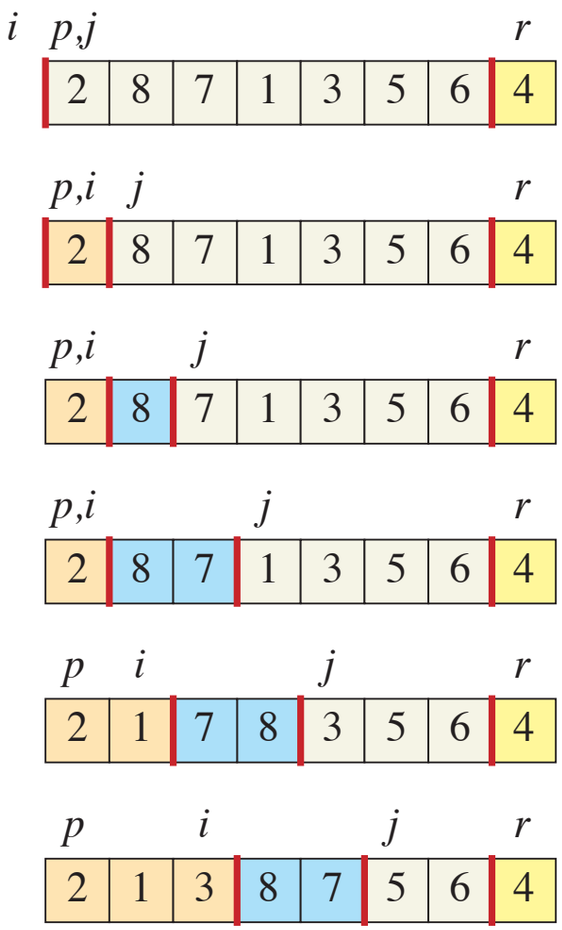
Value *2* is swapped with itself, being placed in the *low subarray*.

Values *8, 7* are swapped with themselves, being placed in the *high subarray*.

Value *1* is swapped with value *8*, placed into the *low* & *high subarrays* respectively.

Notice the position within their subarrays is irrelevant.

The PARTITION procedure



The entry at index r will be the pivot.

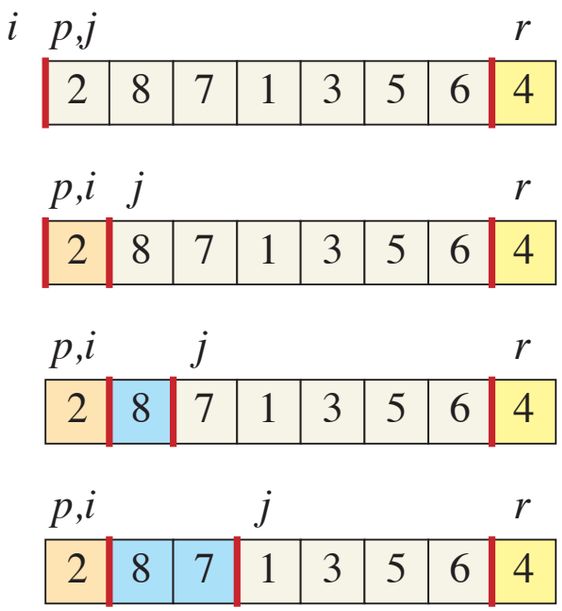
Value *2* is swapped with itself, being placed in the *low subarray*.

Values *8, 7* are swapped with themselves, being placed in the *high subarray*.

Value *1* is swapped with value *8*, placed into the *low* & *high subarrays* respectively.

Value *3* is swapped with value *7*, placed into the *low* & *high subarrays* respectively.

The PARTITION procedure



The entry at index r will be the pivot.

Value *2* is swapped with itself, being placed in the *low subarray*.

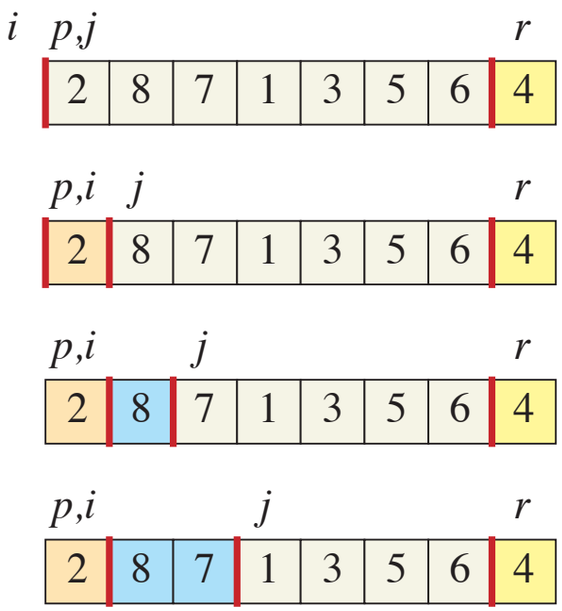
Values *8, 7* are swapped with themselves, being placed in the *high subarray*.

Value *1* is swapped with value *8*, placed into the *low* & *high subarrays* respectively.

Value *3* is swapped with value *7*, placed into the *low* & *high subarrays* respectively.

Continue such ...

The PARTITION procedure



The entry at index *r* will be the pivot.

Value *2* is swapped with itself, being placed in the *low subarray*.

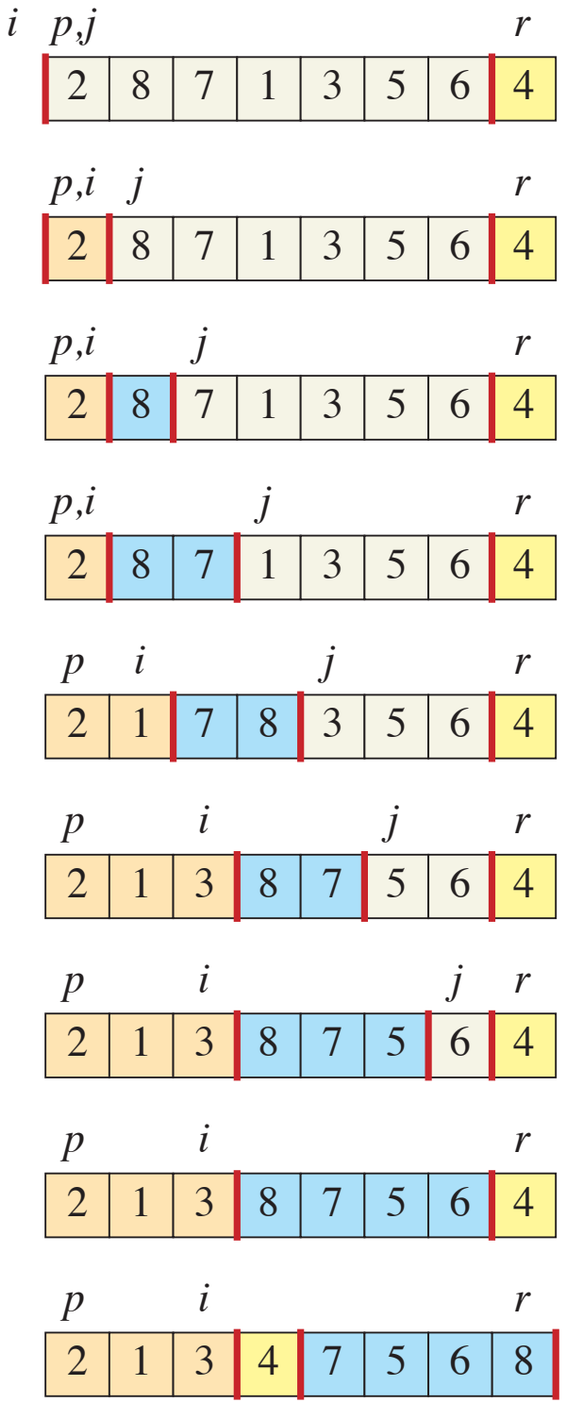
Values *8, 7* are swapped with themselves, being placed in the *high subarray*.

Value *1* is swapped with value *8*, placed into the *low* & *high subarrays* respectively.

Value *3* is swapped with value *7*, placed into the *low* & *high subarrays* respectively.

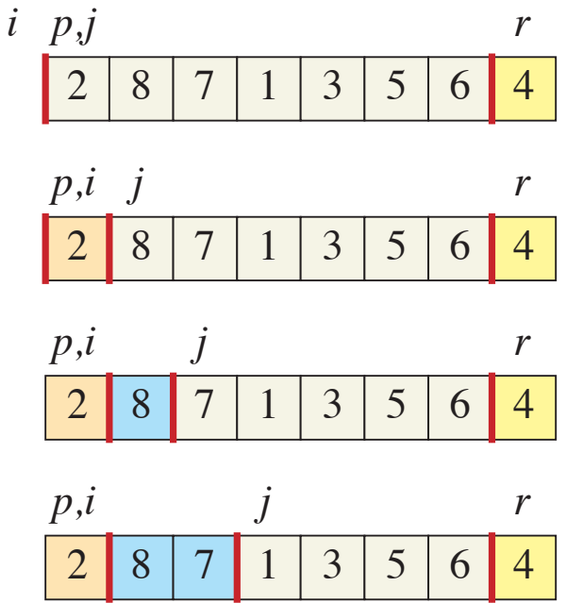
Continue such ... Till the pivot is moved.

The PARTITION procedure



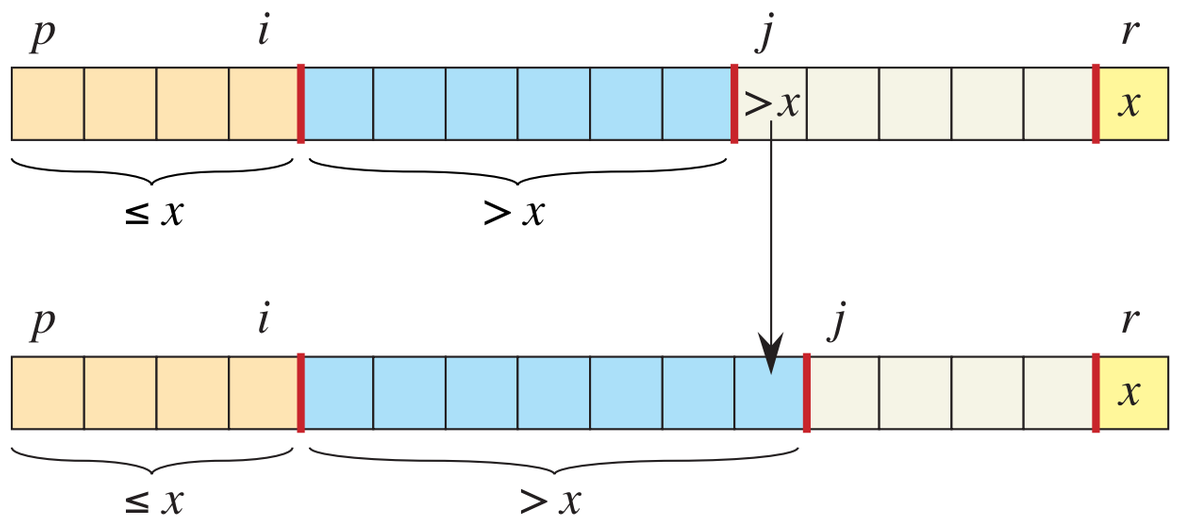
The partition procedure, decides between two possible moves:

The PARTITION procedure



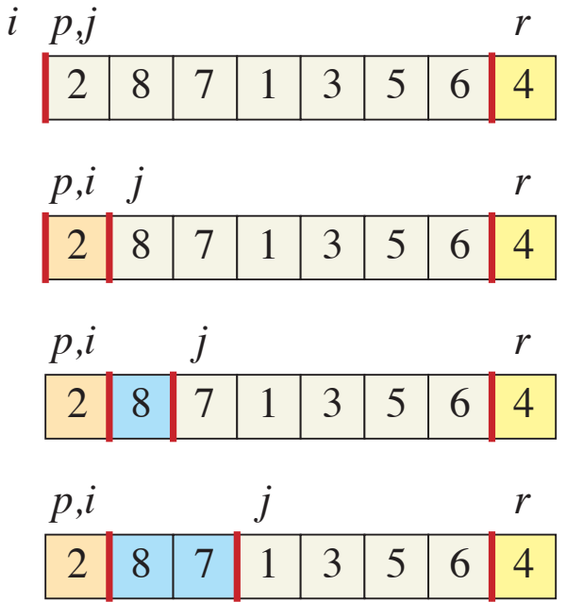
The partition procedure, decides between two possible moves:

- When a value is swapped with itself, to grow the *high subarray*



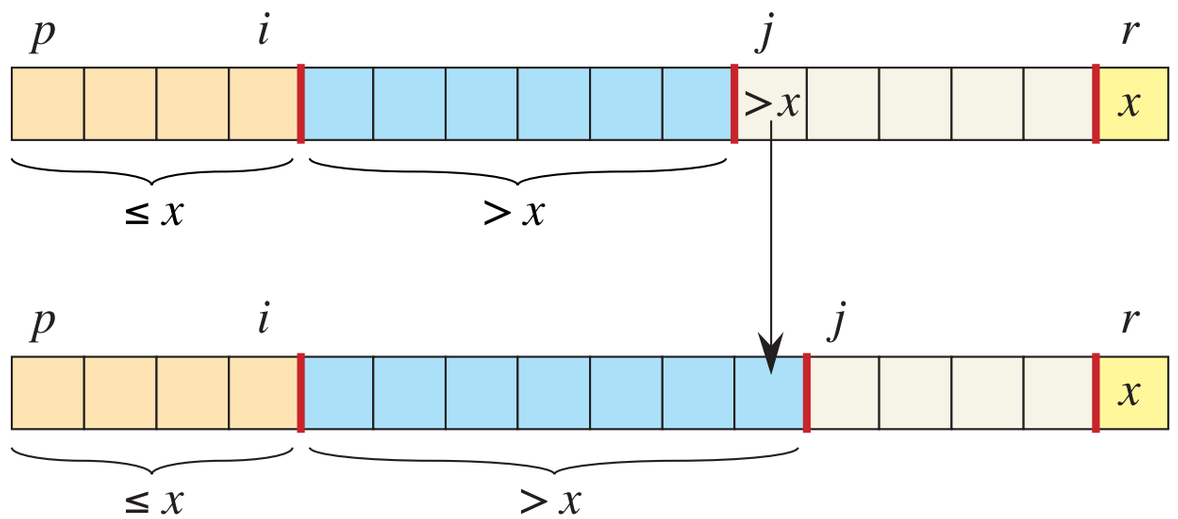
The only required action is increasing index j.

The PARTITION procedure



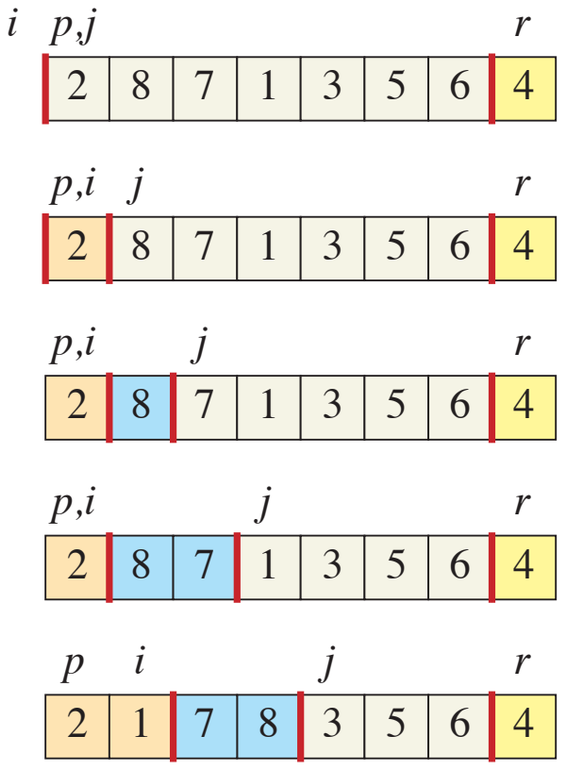
The partition procedure, decides between two possible moves:

- When a value is swapped with itself, to grow the *high subarray* *



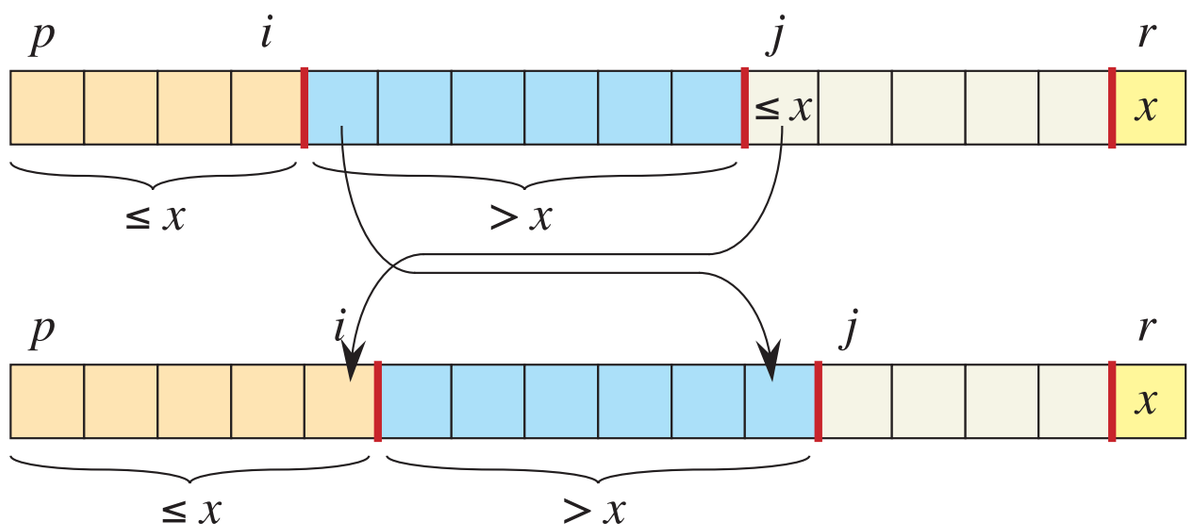
The only required action is increasing index *j*.

The PARTITION procedure



The partition procedure, decides between two possible moves:

- When a value is swapped with itself, to grow the *high subarray* *
- When a value is swapped with another, to grow the *low array*.



Indices i & j are incremented, and the keys are swapped.

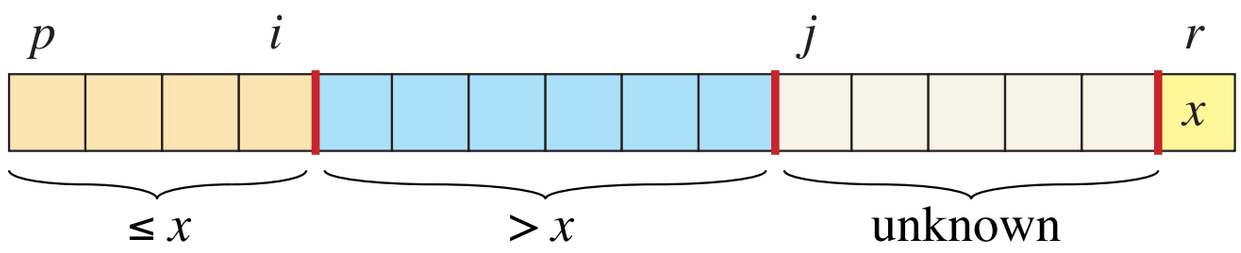
The PARTITION procedure

- Consider an array A, with entries at indexes $p, \ldots, r$ to be sorted.
- We wish to partition the array in place into two subarrays around a pivot (which will be moved, but we will track it).

```
PARTITION(A, p, r)
1  x = A[r]                          // the pivot
2  i = p − 1                         // highest index into the low side
3  for j = p to r − 1                // process each element other than the pivot
4      if A[j] ≤ x                   // does this element belong on the low side?
5          i = i + 1                     // index of a new slot in the low side
6          exchange A[i] with A[j]       // put this element there
7  exchange A[i + 1] with A[r]       // pivot goes just to the right of the low side
8  return i + 1                      // new index of the pivot
```

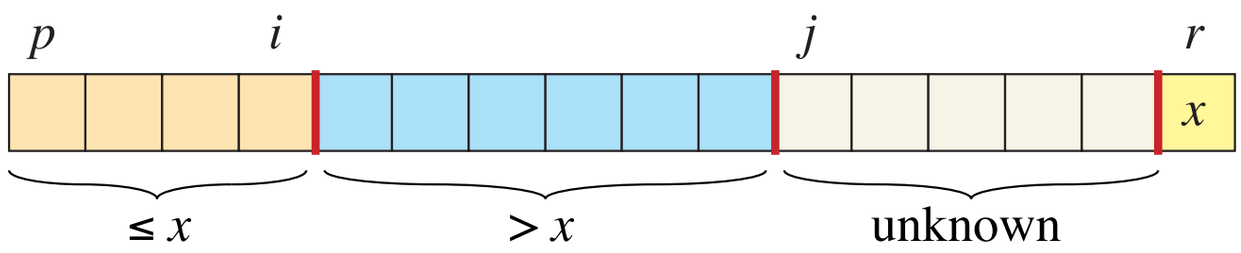
PARTITION loop invariant

- The partition procedure admits the following loop invariant:
  - If $p \le k \le i$ then $A[k] \le x$
  - If $i + 1 \le k \le j - 1$ then $A[k] > x$
  - If $k = r$ then $A[k] = x$



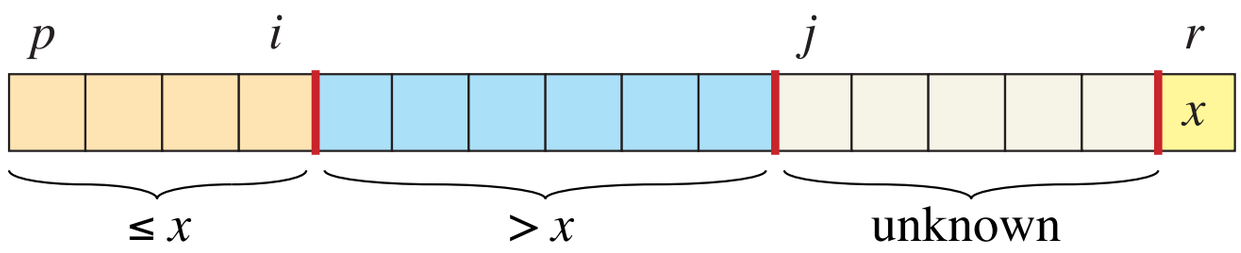
PARTITION loop invariant

- The partition procedure admits the following loop invariant:
  - If $p \le k \le i$ then $A[k] \le x$
  - If $i + 1 \le k \le j - 1$ then $A[k] > x$
  - If $k = r$ then $A[k] = x$



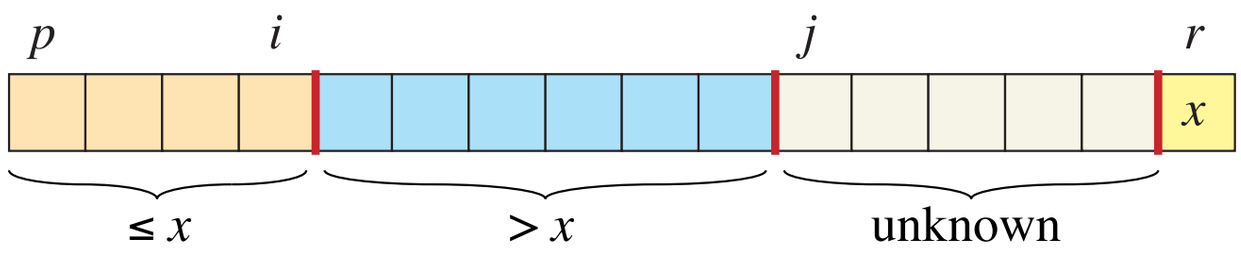
PARTITION loop invariant

- The partition procedure admits the following loop invariant:
  - If $p \le k \le i$ then $A[k] \le x$
  - If $i + 1 \le k \le j - 1$ then $A[k] > x$
  - If $k = r$ then $A[k] = x$



PARTITION loop invariant

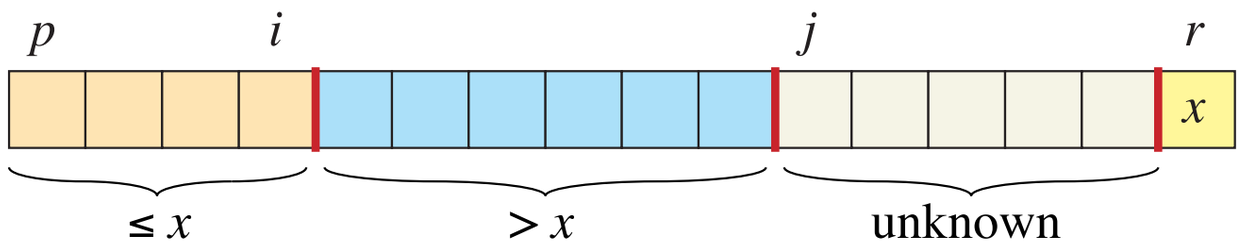
- The partition procedure admits the following loop invariant:
  - If $p \le k \le i$ then $A[k] \le x$
  - If $i + 1 \le k \le j - 1$ then $A[k] > x$
  - If $k = r$ then $A[k] = x$



PARTITION loop invariant

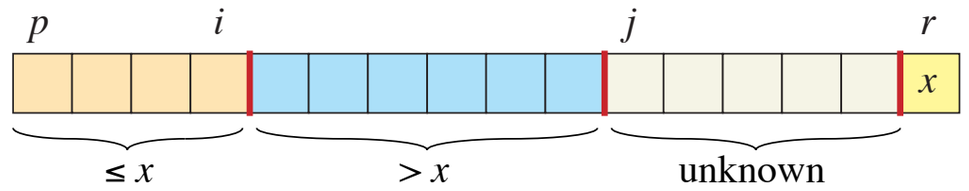
- The partition procedure admits the following loop invariant:
  - If $p \le k \le i$ then $A[k] \le x$
  - If $i + 1 \le k \le j - 1$ then $A[k] > x$
  - If $k = r$ then $A[k] = x$

This should hold for any array index k, at the beginning of each for loop iteration.



PARTITION loop invariant

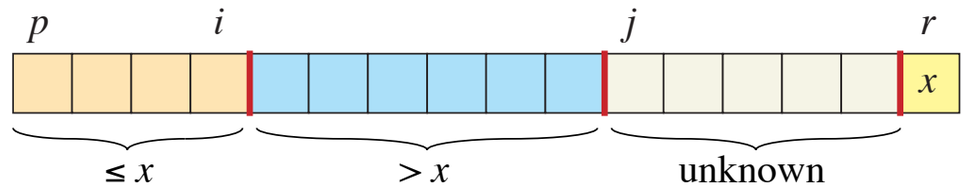
- Let us check the loop invariant holds.




```
PARTITION(A, p, r)
1   x = A[r]
2   i = p − 1
3   for j = p to r − 1
4       if A[j] ≤ x
5           i = i + 1
6           exchange A[i] with A[j]
7   exchange A[i + 1] with A[r]
8   return i + 1
```

PARTITION loop invariant

- Let us check the loop invariant holds.

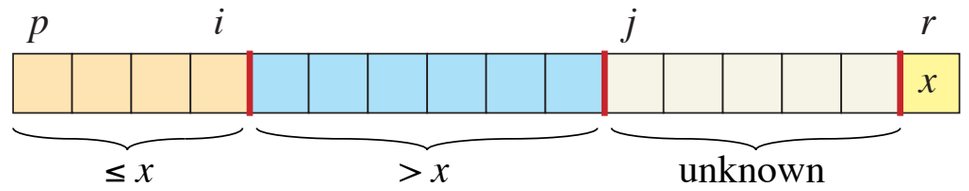



```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Initialization:

PARTITION loop invariant

- Let us check the loop invariant holds.

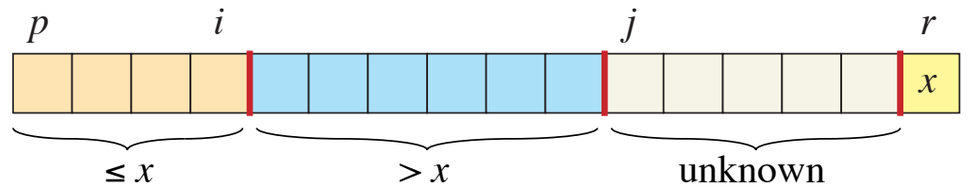



```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Maintenance:

PARTITION loop invariant

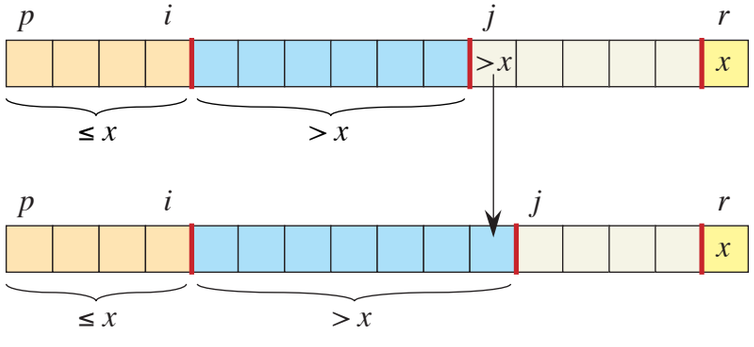
- Let us check the loop invariant holds.



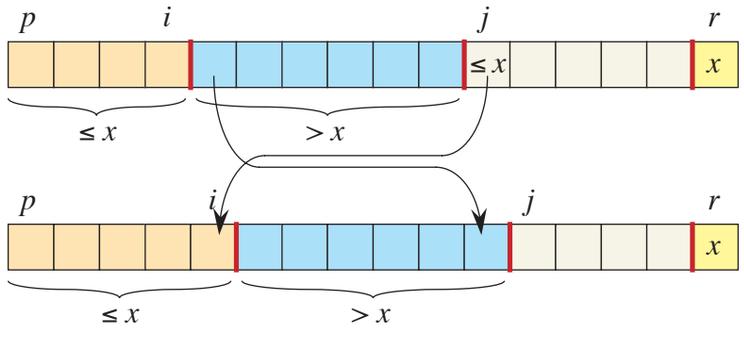

```
PARTITION(A, p, r)
1   x = A[r]
2   i = p − 1
3   for j = p to r − 1
4       if A[j] ≤ x
5           i = i + 1
6           exchange A[i] with A[j]
7   exchange A[i + 1] with A[r]
8   return i + 1
```

Maintenance:

Case 1

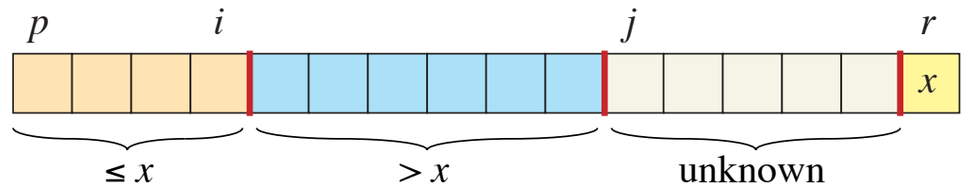


Case 2



PARTITION loop invariant

- Let us check the loop invariant holds.




```
PARTITION(A, p, r)
1   x = A[r]
2   i = p − 1
3   for j = p to r − 1
4       if A[j] ≤ x
5           i = i + 1
6           exchange A[i] with A[j]
7   exchange A[i + 1] with A[r]
8   return i + 1
```

Termination:

Exercise

- What is the running time for the Partition operation?

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4       if A[j] ≤ x
5            i = i + 1
6            exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

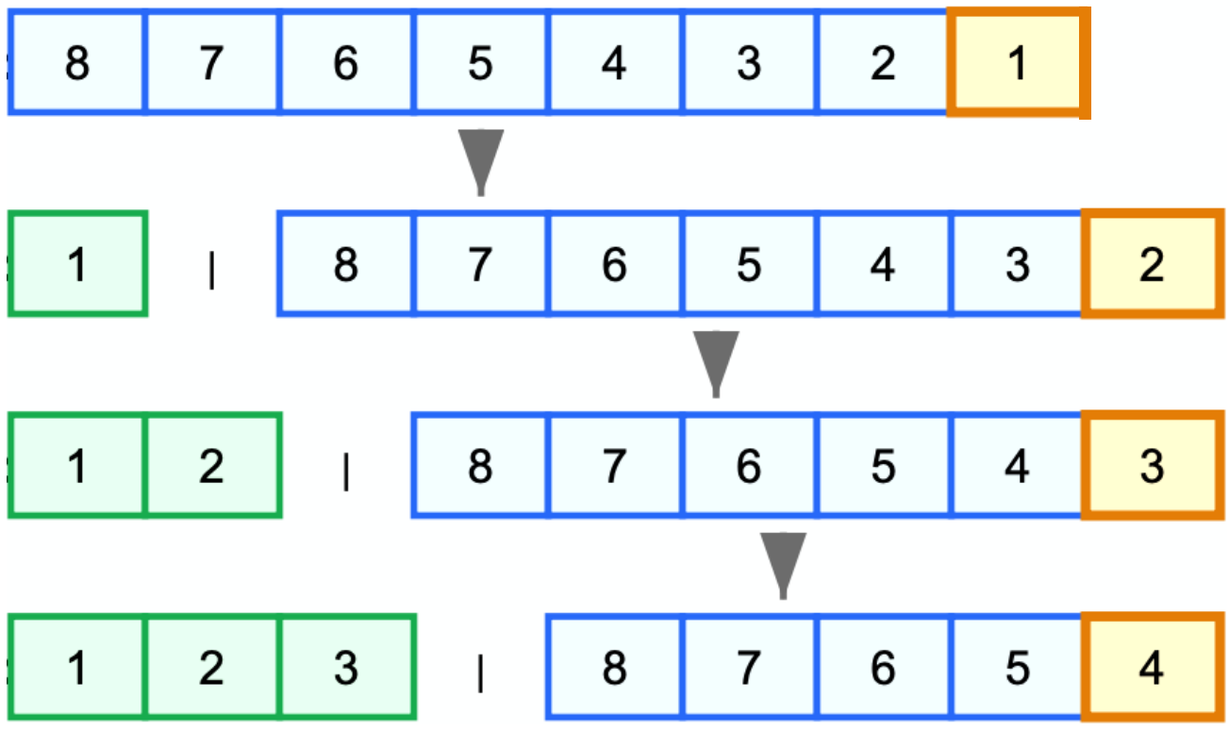
$\Theta(n)$ for an array of size n.

Since we run through the array, making one comparison check per element.

Quick sort performance

- The running time of Quick sort will depend on how balanced each partitioning is.
    - **Worst case partitioning:**

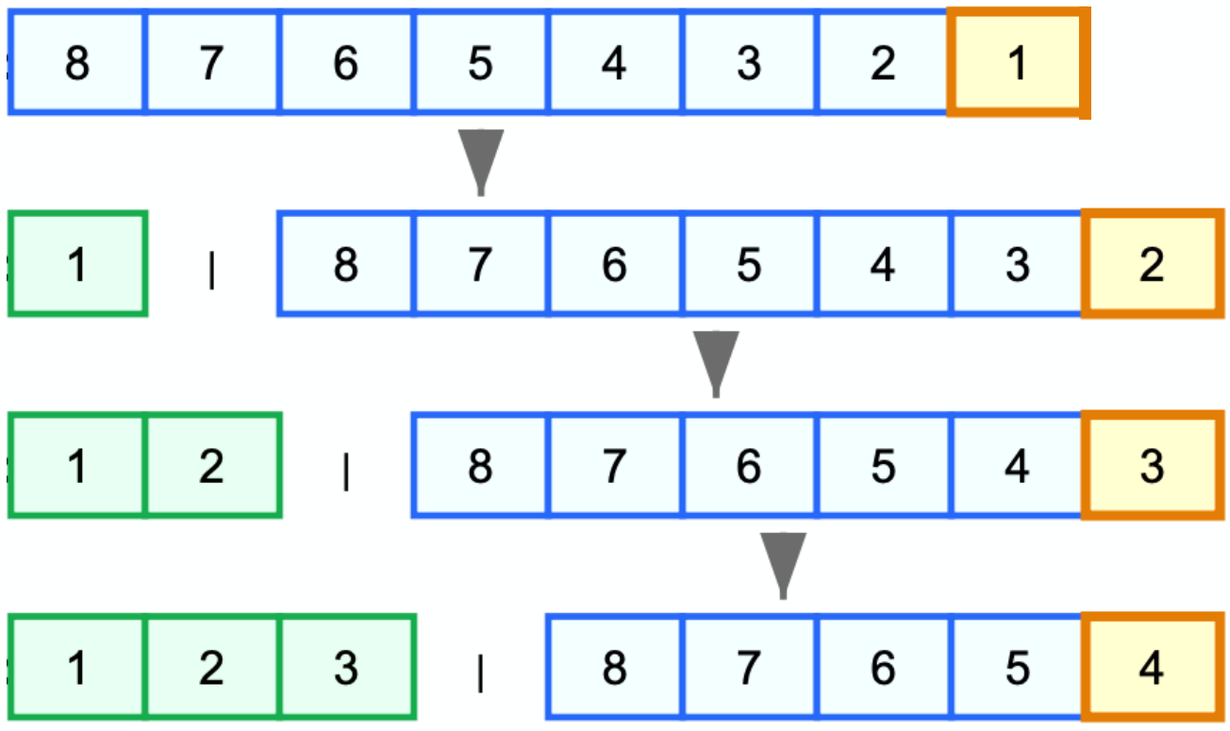
      One subarray with *n-1* elements, one with *0* and the pivot.



Quick sort performance

- The running time of Quick sort will depend on how balanced each partitioning is.
    - **Worst case partitioning:**

      One subarray with *n-1* elements, one with *0* and the pivot.

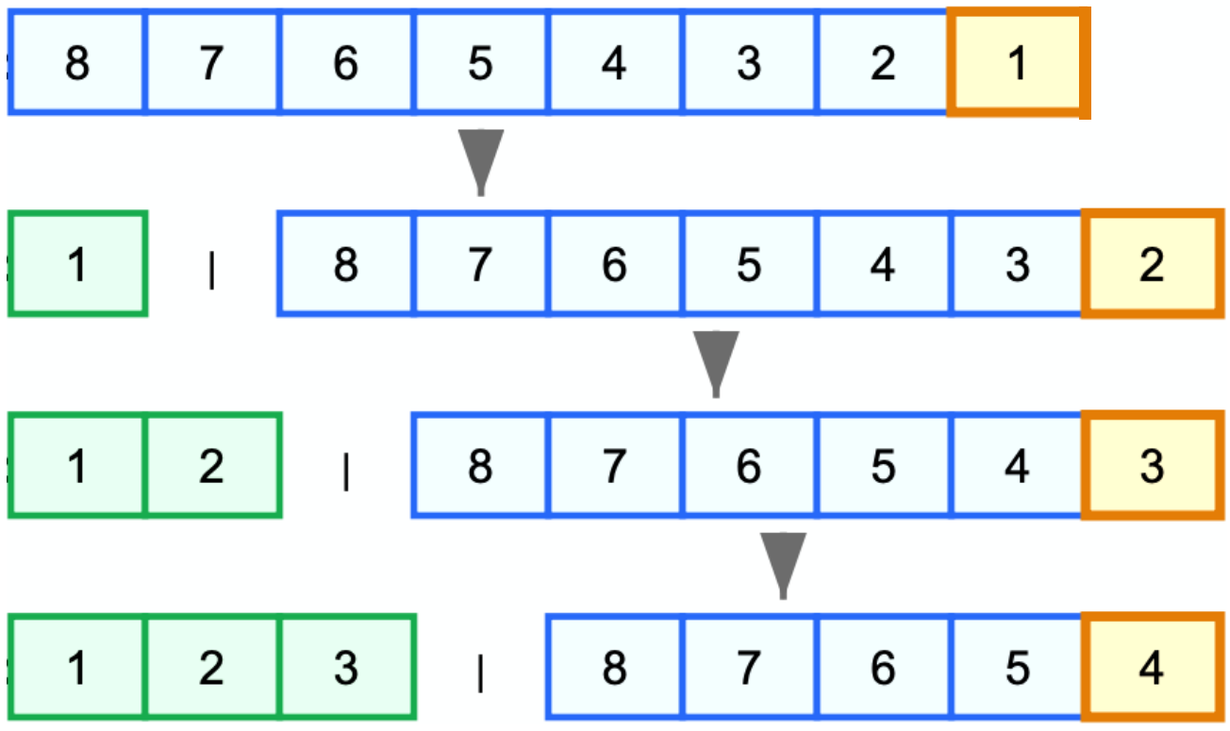


$$T(n) = T(n-1) + T(0) + \Theta(n)$$
$$= T(n-1) + \Theta(n)$$
$$= \Theta(n^2)$$

QUICK SORT performance

- The running time of Quick sort will depend on how balanced each partitioning is.
    - **Worst case partitioning:**

      One subarray with *n-1* elements, one with *0* and the pivot.



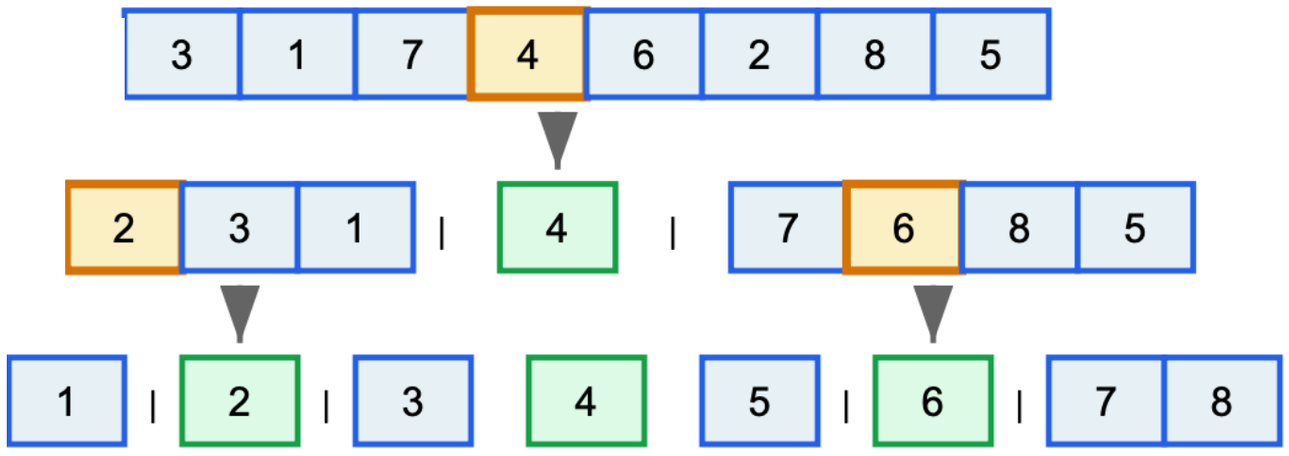
$$T(n) = T(n-1) + T(0) + \Theta(n)$$
$$= T(n-1) + \Theta(n)$$
$$= \Theta(n^2)$$

Similar to INSERTION SORT.

Quick sort performance

- The running time of Quick sort will depend on how balanced each partitioning is.
    - **Best case partitioning:**

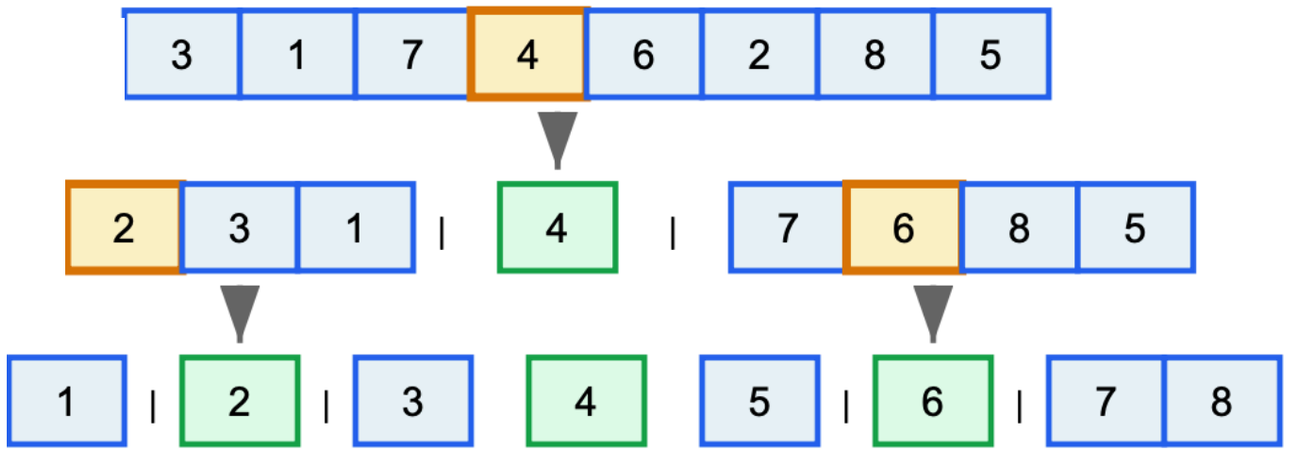
      Each subarray split in half at each iteration.



Quick sort performance

- The running time of Quick sort will depend on how balanced each partitioning is.
    - **Best case partitioning:**

      Each subarray split in half at each iteration.



$$T(n) = 2T(n/2) + T(0) + \Theta(n)$$

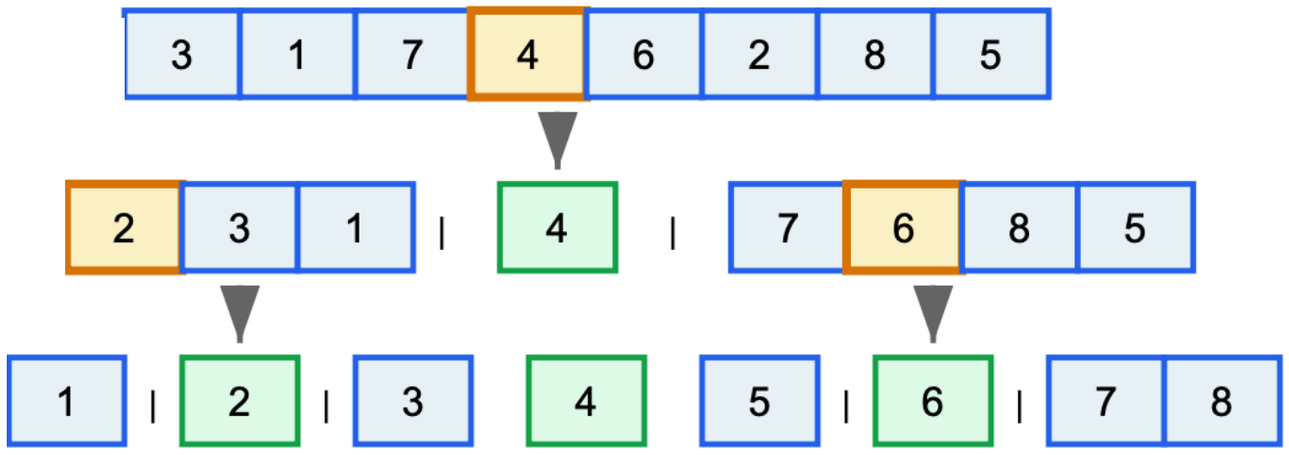
$$= T(n/2) + \Theta(n)$$

$$= \Theta(nlog(n))$$

Quick sort performance

- The running time of Quick sort will depend on how balanced each partitioning is.
    - **Best case partitioning:**

      Each subarray split in half at each iteration.



$$T(n) = 2T(n/2) + T(0) + \Theta(n)$$
$$= T(n/2) + \Theta(n)$$
$$= \Theta(nlog(n))$$

Similar to Merge Sort

Homework (due with Assignment 1)

- Banks record transactions with respect to the time of the transaction, but customers like to receive their bank statements with checks listed in order by check number. As a checkbook is ordered by check number, people tend to use their checks in order by check number. Furthermore, merchants usually cash checks reasonably promptly.

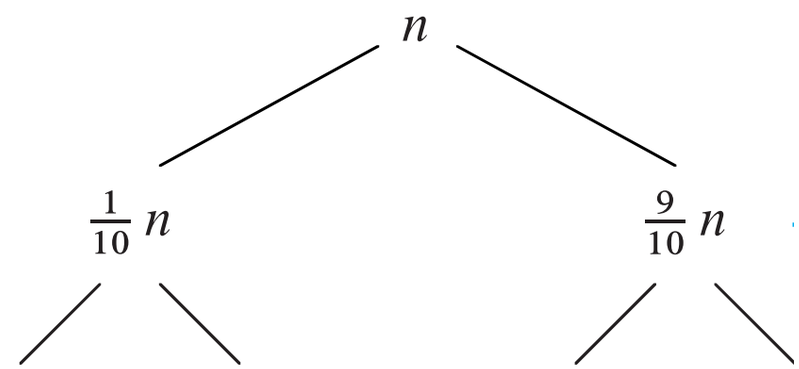
  Thus, the problem of converting time-of-transaction ordering to check-number ordering corresponds to the problem of sorting almost-sorted input.

  Explain persuasively why the INSERTION SORT algorithm might tend to beat QUICK SORT on this problem.

QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

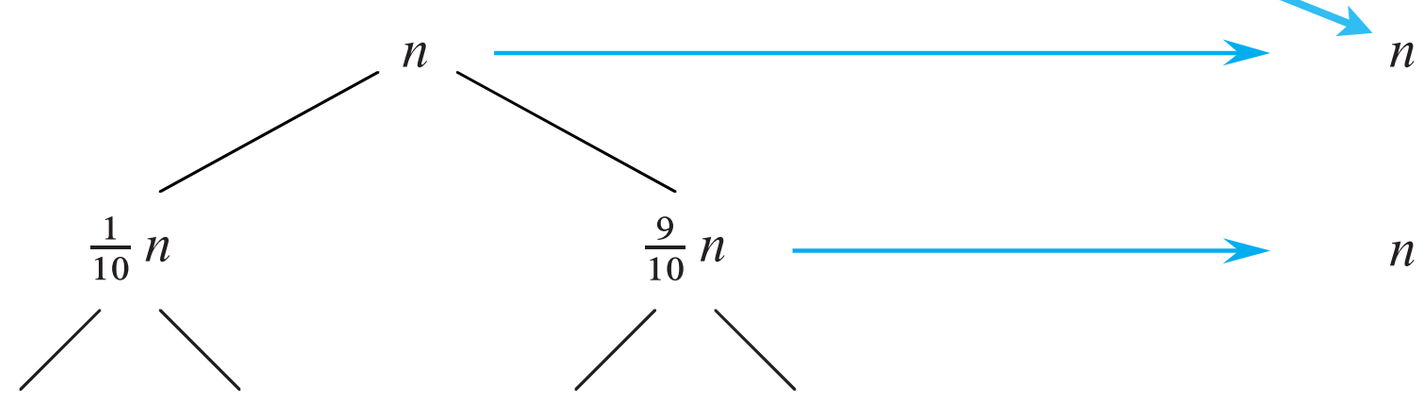
$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

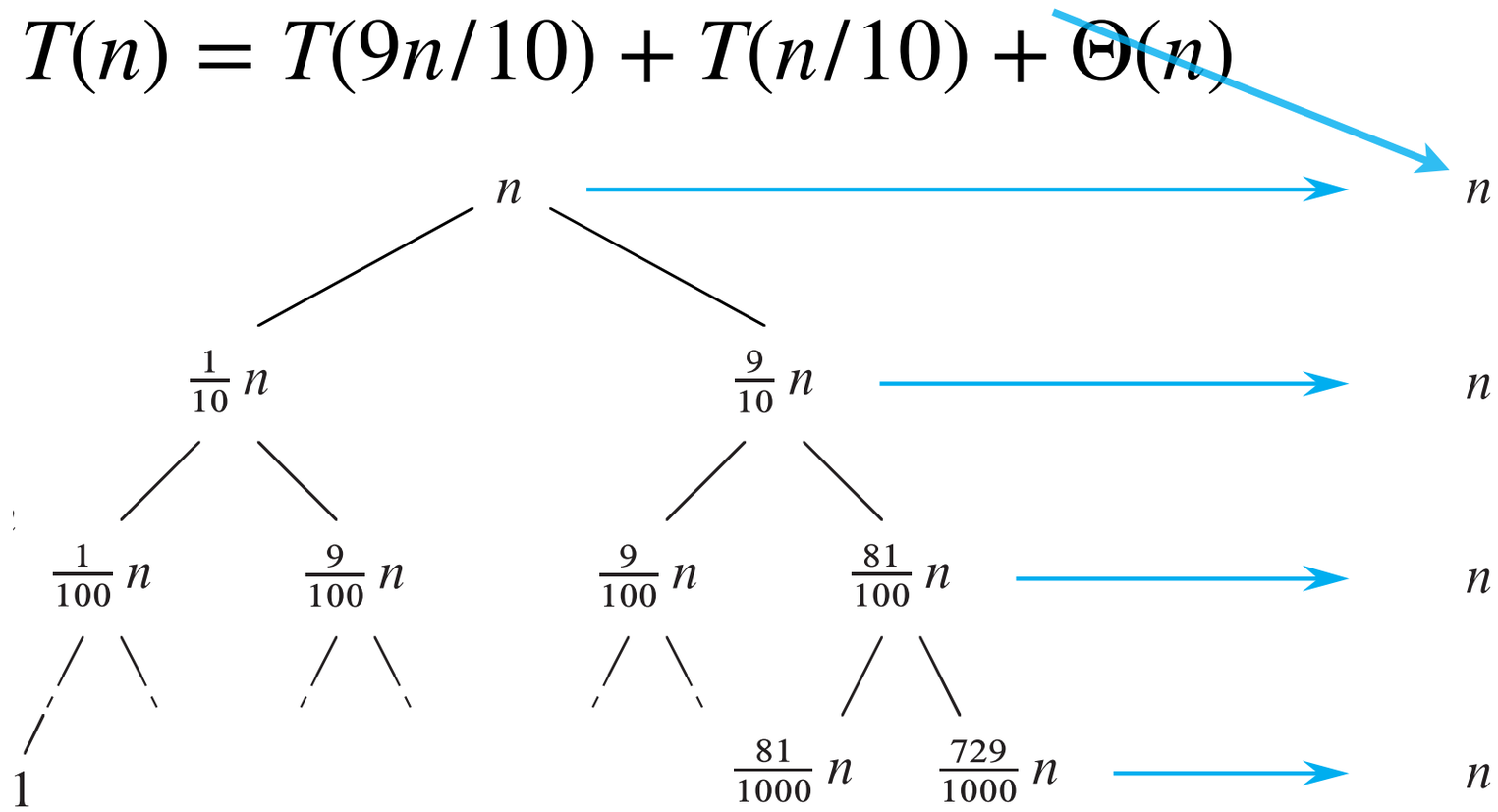
$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

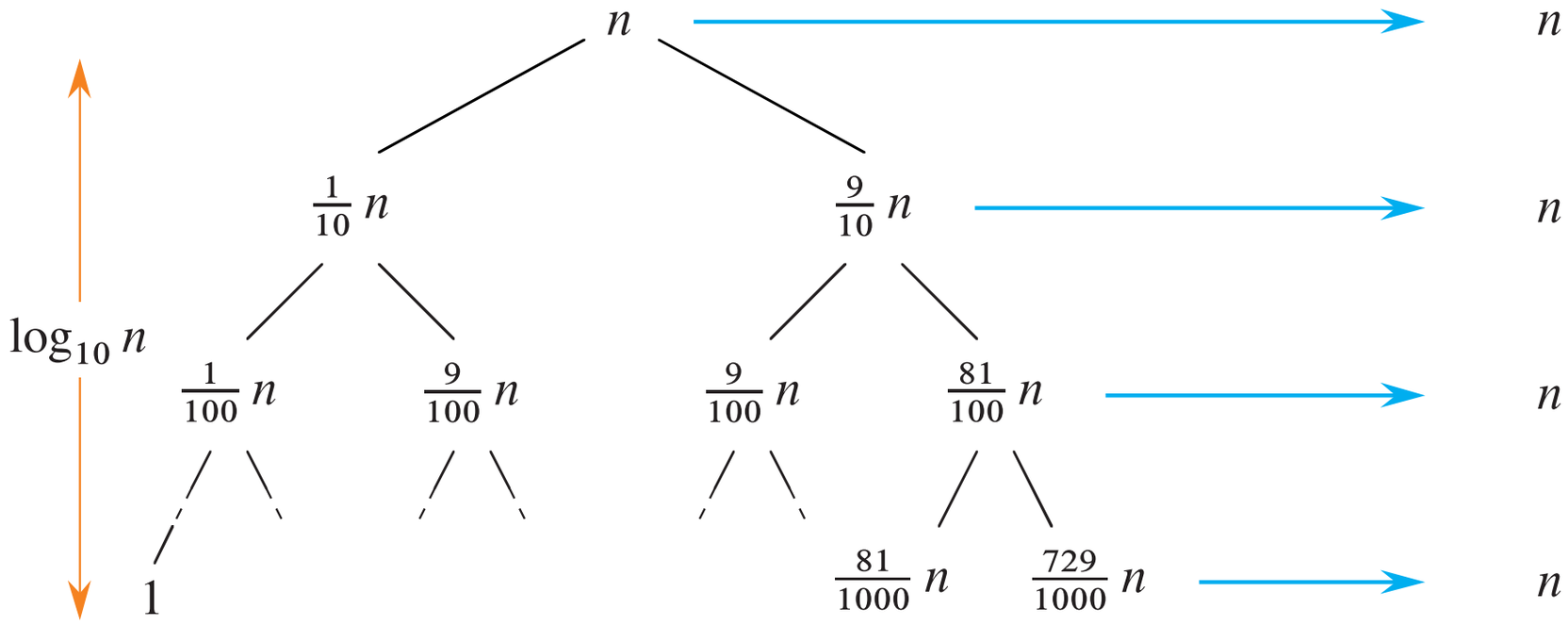
$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

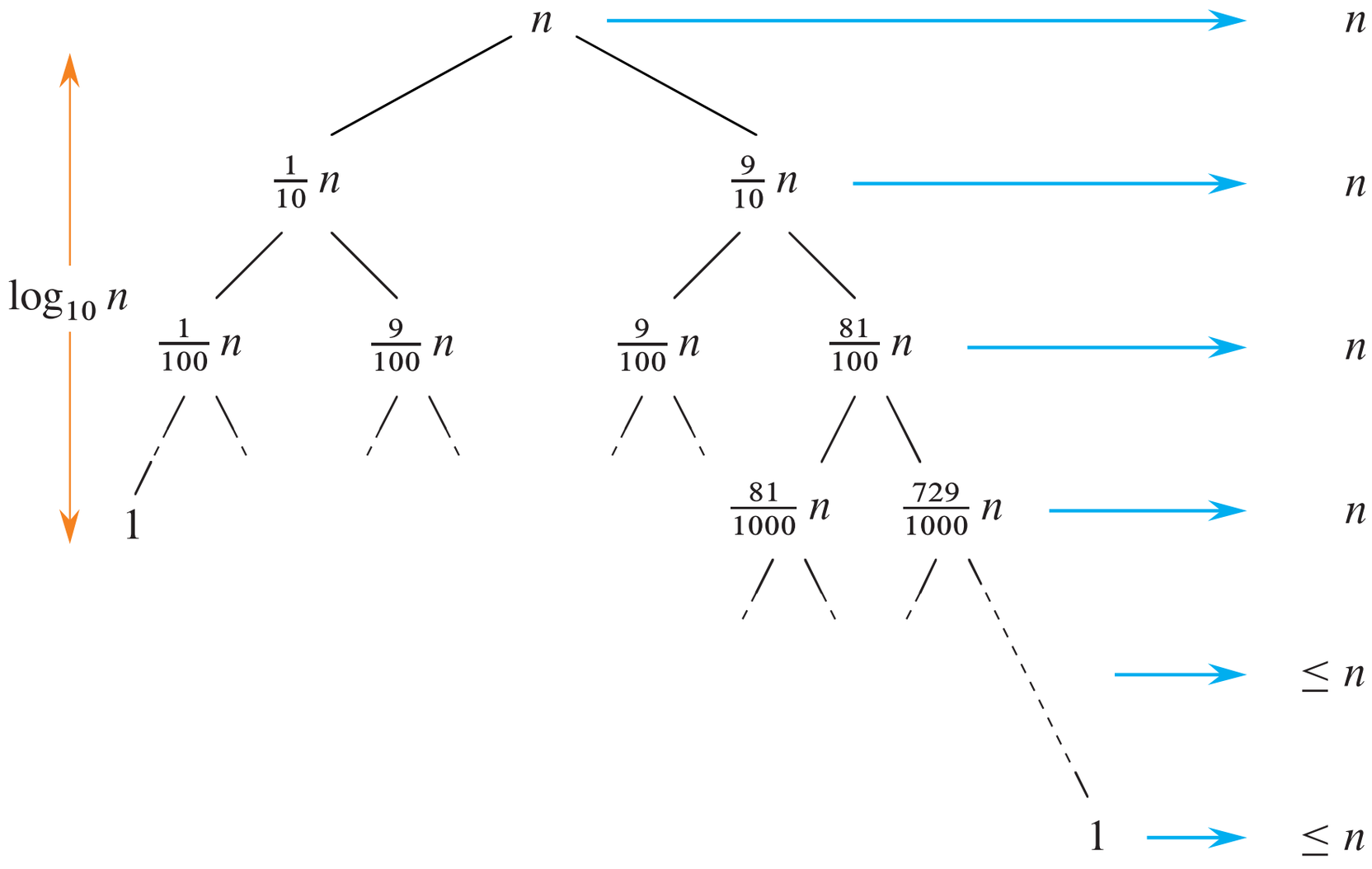
$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

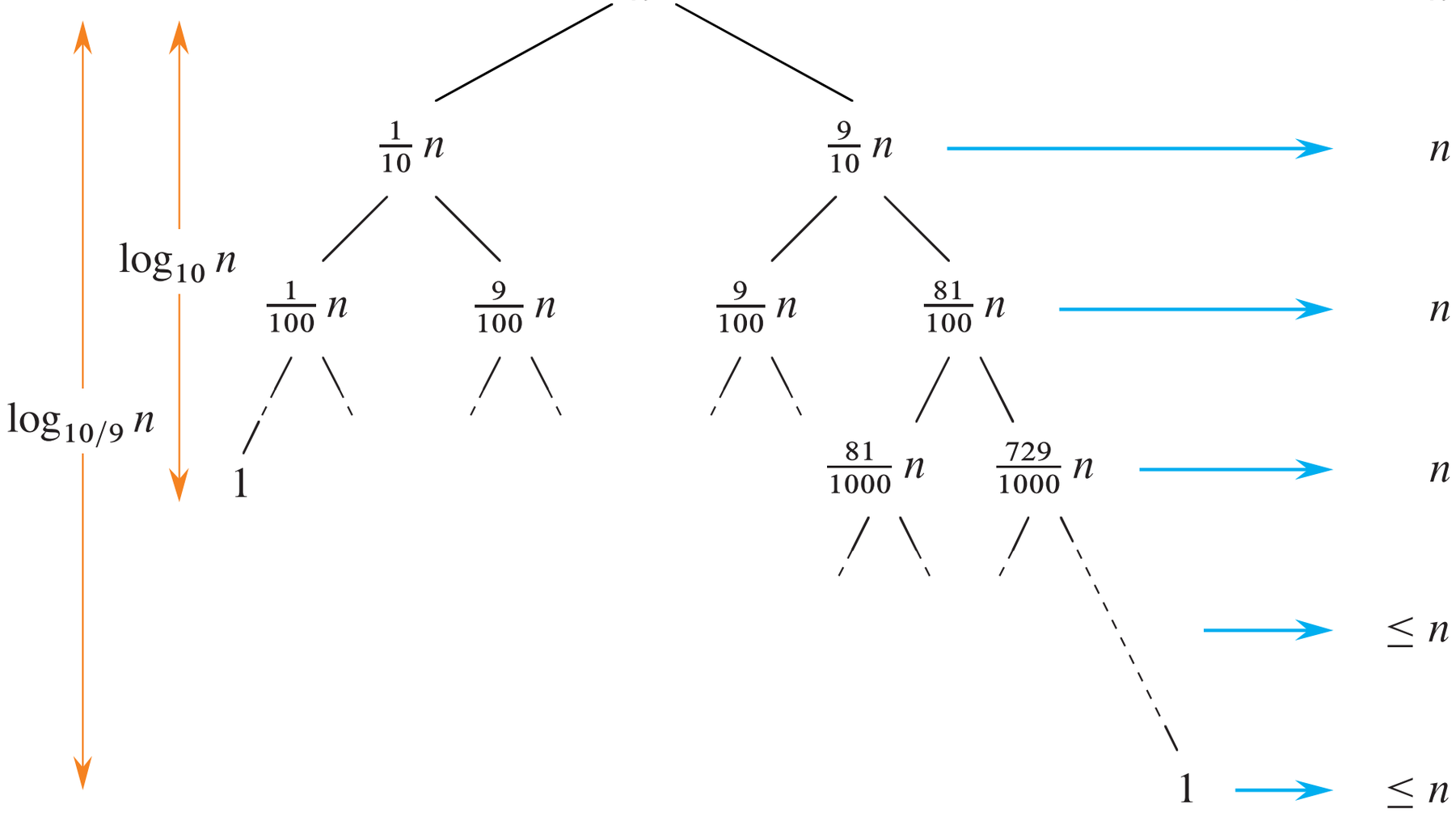
$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

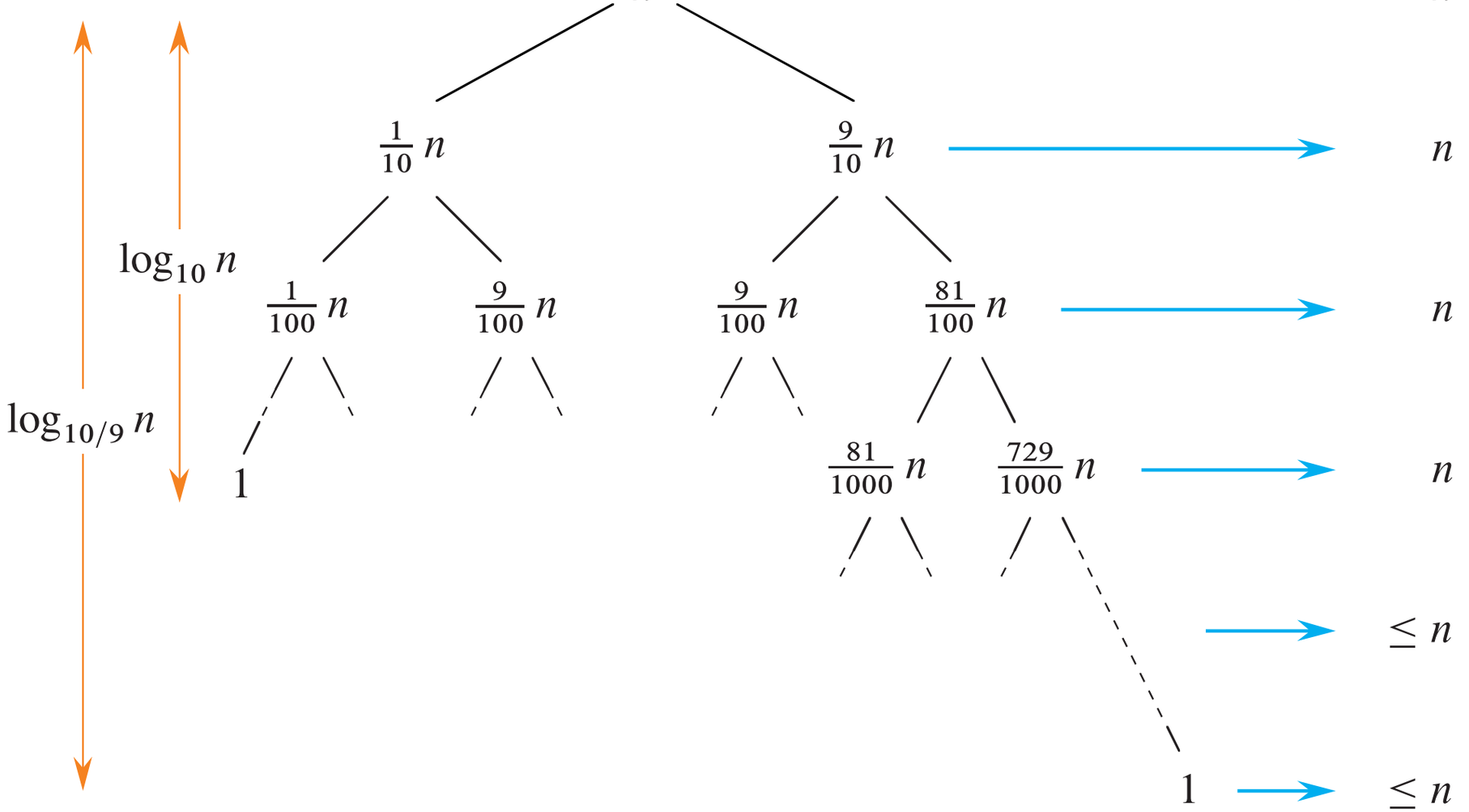
$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$

- **Exercise:** Show you can guarantee a 9 to 1 partition for about 80% of partitions, assuming a random distribution of the entries.

QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$

- **Exercise:** Show you can guarantee a 9 to 1 partition for about 80% of partitions, assuming a random distribution of the entries.
- Assume your QUICKSORT algorithm runs with an alternation of "good" and "bad" splits. What would be the running time?

QUICK SORT average performance

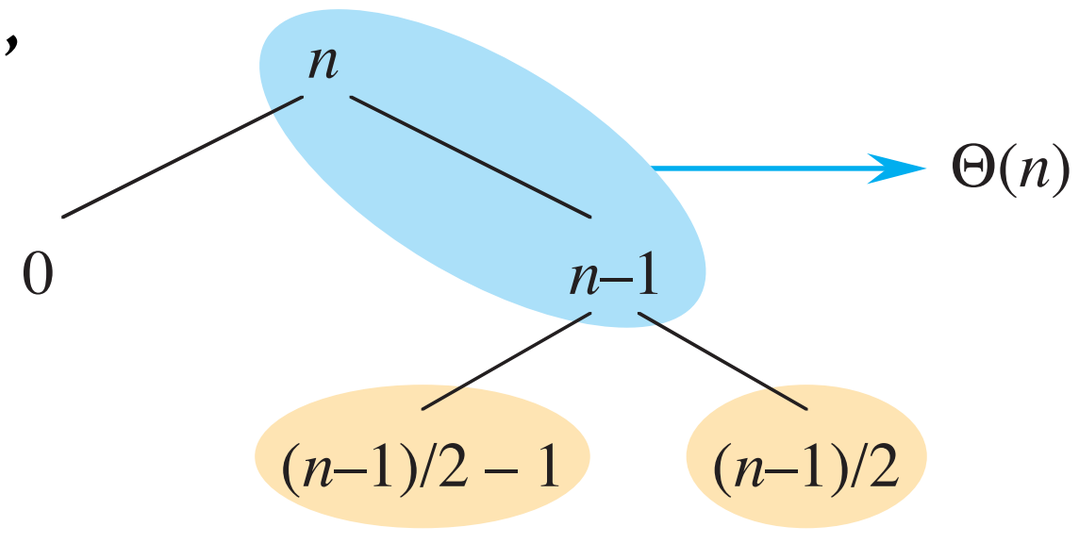
- Let us compute the running time for a **9 to 1 partition**

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$

- **Exercise:** Show you can guarantee a 9 to 1 partition for about 80% of partitions, assuming a random distribution of the entries.

- Assume your QUICKSORT algorithm runs with an alternation of "good" and "bad" splits. What would be the running time?

- For simplicity "good"= best case, "bad"=worst case partition.



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

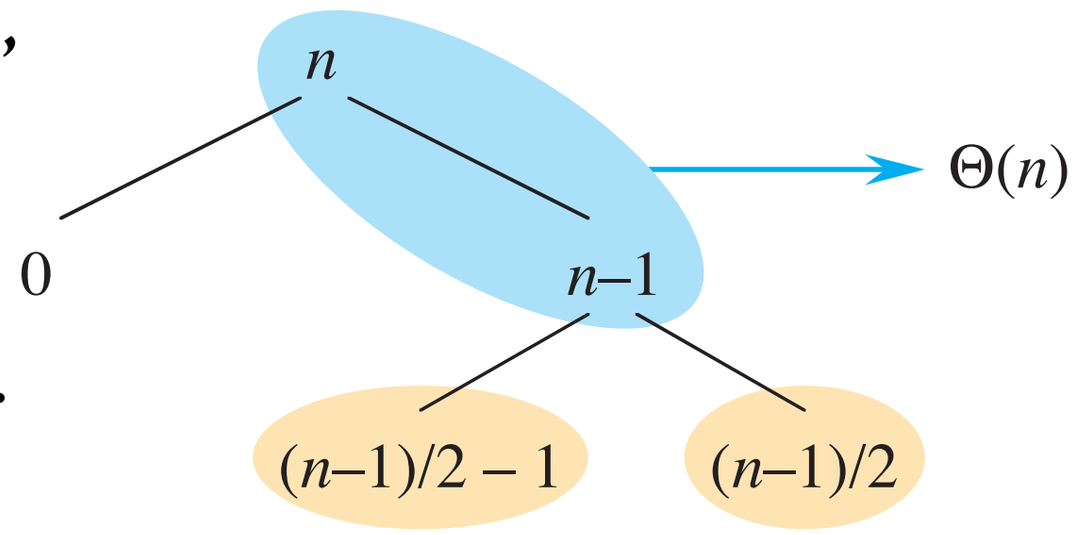
$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$

- **Exercise:** Show you can guarantee a 9 to 1 partition for about 80% of partitions, assuming a random distribution of the entries.

- Assume your QUICKSORT algorithm runs with an alternation of "good" and "bad" splits. What would be the running time?

- For simplicity "good"= best case, "bad"=worst case partition.

- Doing two such splits, is similar to a single split, in the best case.



QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$

- **Exercise:** Show you can guarantee a 9 to 1 partition for about 80% of partitions, assuming a random distribution of the entries.

- Assume your QUICKSORT algorithm runs with an alternation of "good" and "bad" splits. What would be the running time?

- For simplicity "good"= best case, "bad"=worst case partition.

- Doing two such splits, is similar to a single split, in the best case.

We conclude doing one good and one bad split, is about twice as slow as doing only good splits.

QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$

- **Exercise:** Show you can guarantee a 9 to 1 partition for about 80% of partitions, assuming a random distribution of the entries.
- Assume your QUICKSORT algorithm runs with an alternation of "good" and "bad" splits. What would be the running time?

$$T(n) = \alpha \cdot O(nlogn) = O(nlogn)$$

QUICK SORT average performance

- Let us compute the running time for a **9 to 1 partition**

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = \Theta(nlog_{10/9}(n))$$

- **Exercise:** Show you can guarantee a 9 to 1 partition for about 80% of partitions, assuming a random distribution of the entries.
- Assume your QUICKSORT algorithm runs with an alternation of "good" and "bad" splits. What would be the running time?

$$T(n) = \alpha \cdot O(nlogn) = O(nlogn)$$

- Assuming a random distribution of entries, the average running time of QUICKSORT is $O(nlogn)$.

Homework (due with Assignment 1)

- What is the running time of QUICK SORT when all elements of the array have the same value?

Randomized QUICK SORT

- Since QUICKSORT tend to work best with randomized entries, we might consider randomizing the entries first.

Randomized QUICK SORT

- Since QUICKSORT tend to work best with randomized entries, we might consider randomizing the entries first.
- A simpler solution - to randomly select the pivot.

Randomized QUICK SORT

- Since QUICKSORT tend to work best with randomized entries, we might consider randomizing the entries first.
- A simpler solution - to randomly select the pivot.
- By choosing the pivot randomly, we can expect the split of the array to be reasonably well balanced.

Randomized QUICK SORT

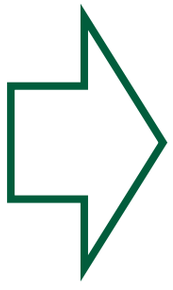
- Since QUICKSORT tend to work best with randomized entries, we might consider randomizing the entries first.
- A simpler solution - to randomly select the pivot.
- By choosing the pivot randomly, we can expect the split of the array to be reasonably well balanced.

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4       if A[j] ≤ x
5            i = i + 1
6            exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Randomized QUICK SORT

- Since QUICKSORT tend to work best with randomized entries, we might consider randomizing the entries first.
- A simpler solution - to randomly select the pivot.
- By choosing the pivot randomly, we can expect the split of the array to be reasonably well balanced.

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4       if A[j] ≤ x
5            i = i + 1
6            exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```



```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

Exercise

- Compare the two partition functions:

```
PARTITION(A, p, r)
1   x = A[r]
2   i = p − 1
3   for j = p to r − 1
4       if A[j] ≤ x
5           i = i + 1
6           exchange A[i] with A[j]
7   exchange A[i + 1] with A[r]
8   return i + 1
```

```
HOARE-PARTITION(A, p, r)
 1   x = A[p]
 2   i = p − 1
 3   j = r + 1
 4   while TRUE
 5       repeat
 6           j = j − 1
 7       until A[j] ≤ x
 8       repeat
 9           i = i + 1
10       until A[i] ≥ x
11       if i < j
12           exchange A[i] with A[j]
13       else return j
```

Exercise

- Compare the two partition functions:

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

We swap every element smaller than the pivot with the partition boundary.

```
HOARE-PARTITION(A, p, r)
 1  x = A[p]
 2  i = p − 1
 3  j = r + 1
 4  while TRUE
 5      repeat
 6          j = j − 1
 7      until A[j] ≤ x
 8      repeat
 9          i = i + 1
10      until A[i] ≥ x
11      if i < j
12          exchange A[i] with A[j]
13      else return j
```

Exercise

- Compare the two partition functions:

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

We swap every element smaller than the pivot with the partition boundary.

```
HOARE-PARTITION(A, p, r)
 1  x = A[p]
 2  i = p − 1
 3  j = r + 1
 4  while TRUE
 5      repeat
 6          j = j − 1
 7      until A[j] ≤ x
 8      repeat
 9          i = i + 1
10      until A[i] ≥ x
11      if i < j
12          exchange A[i] with A[j]
13      else return j
```

Makes less swaps: ensures both elements moved are on the wrong side.

Exercise

- You are given n red and n blue water jugs, all of different shapes and sizes. All the jugs hold different amounts of water, and you cannot tell from the size of a jug how much water it holds. But, for every jug of one color, there is a jug of the other color that holds the same amount of water.
- Your task is to group the jugs into pairs of red and blue jugs that hold the same amount of water.
- Your goal is to find an algorithm that makes a minimum number of pairwise comparisons between different colored jugs (you are not allowed to compare two red jugs or two blue jugs) to determine the grouping.

Exercise

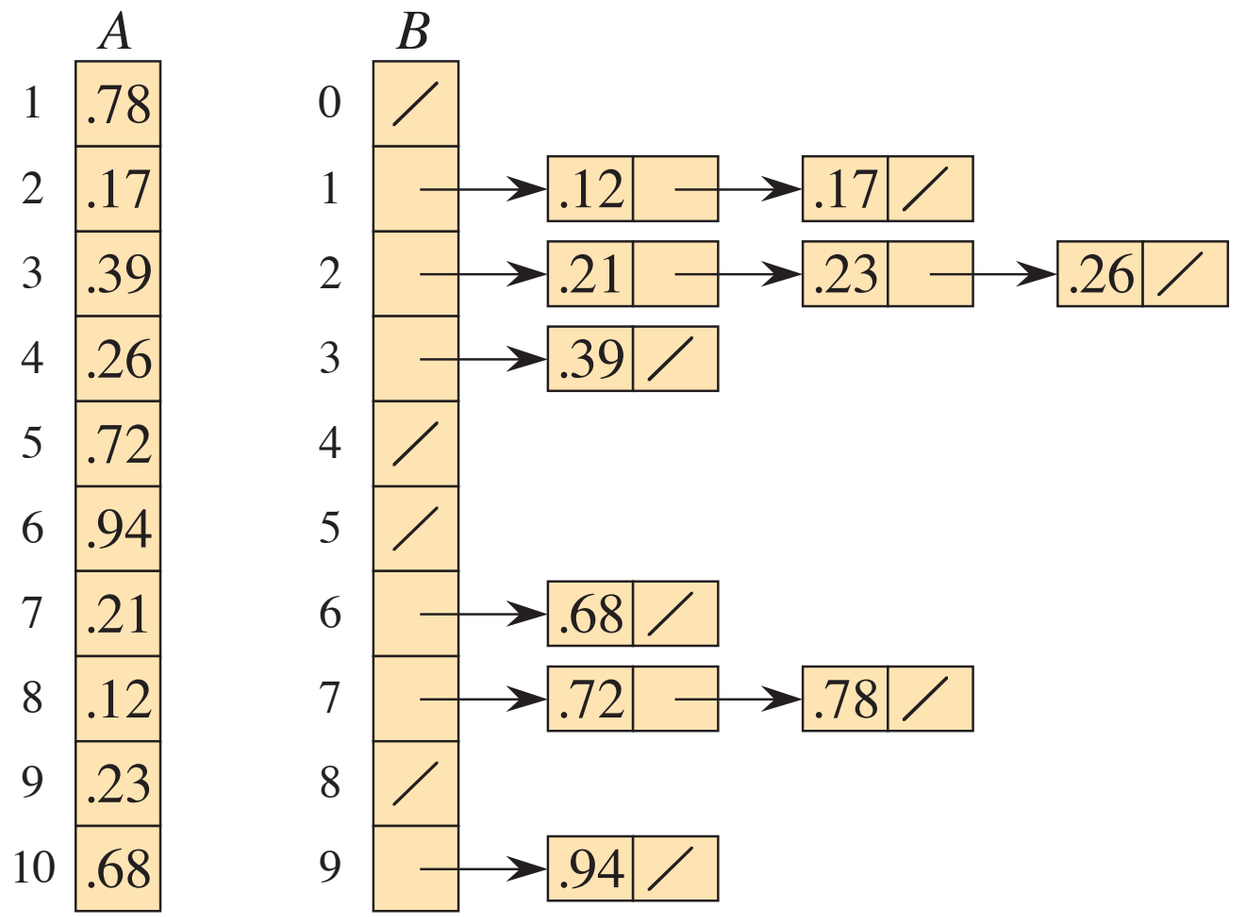
- **Divide:**
  - Pick any red jug R as a pivot
  - Compare R with each blue jug to partition the blue jugs into three groups:
    - Blue jugs smaller than R
    - The blue jug B that equals R (there's exactly one)
    - Blue jugs larger than R
  - Now use the matching blue jug B to partition the remaining red jugs:
    - Red jugs smaller than B
    - Red jugs larger than B
- **Conquer:**
  - Recursively solve the subproblem of smaller and larger jugs
  - The pivot pair (R, B) is already matched

Bucket Sort

- If we can assume the input array has elements that are uniformly distributed, we can design even faster algorithms.
- Bucket sorts assumes the input has values which are uniformly and independently sampled from the interval $[0,1)$.
- It runs in linear time.

Bucket Sort

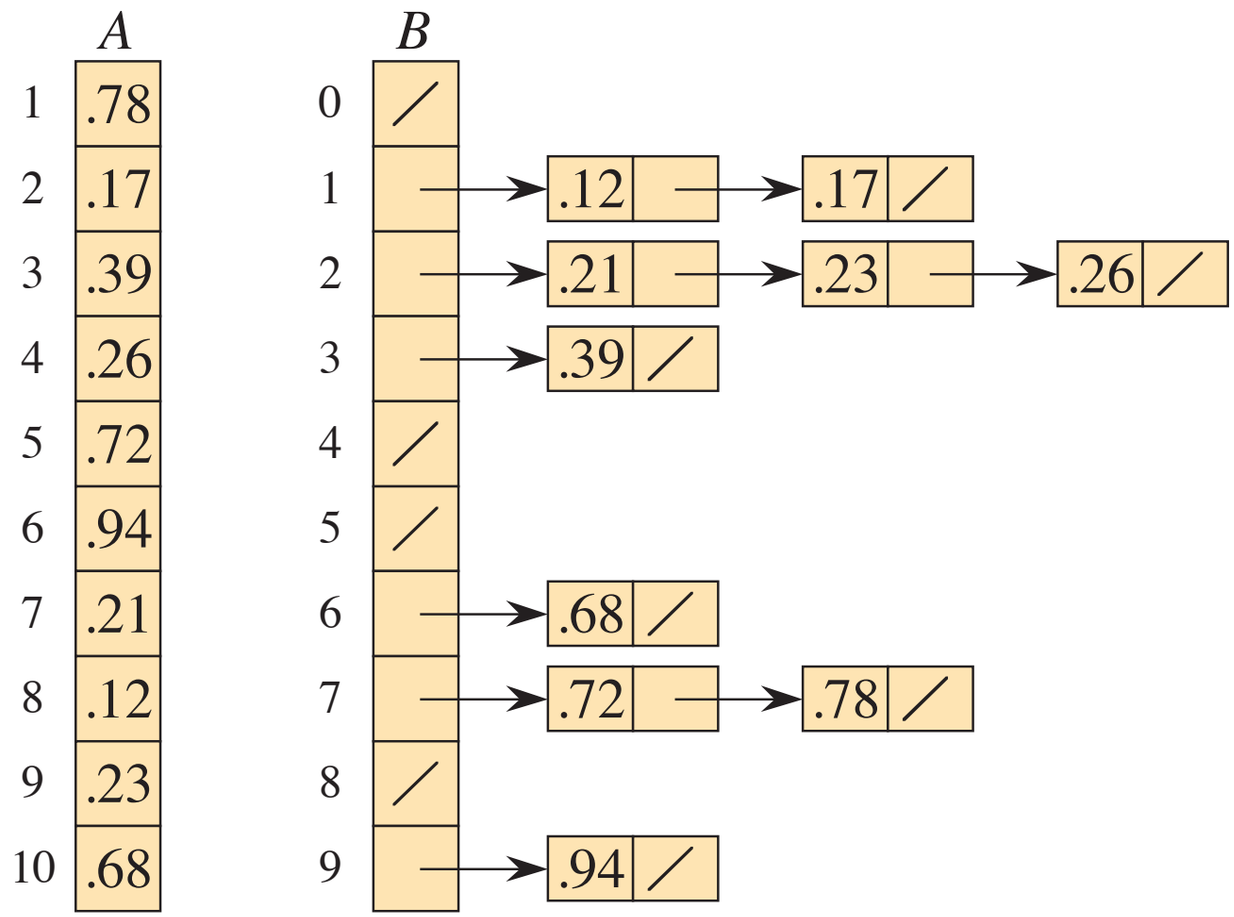
- If we can assume the input array has elements that are uniformly distributed, we can design even faster algorithms.
- Bucket sorts assumes the input has values which are uniformly and independently sampled from the interval $[0,1)$.
- It runs in linear time.



Bucket Sort

- If we can assume the input array has elements that are uniformly distributed, we can design even faster algorithms.
- Bucket sorts assumes the input has values which are uniformly and independently sampled from the interval $[0,1)$.
- It runs in linear time.

Reminiscent of Hash tables :)



BUCKET SORT

```
BUCKET-SORT(A, n)
1  let B[0 : n − 1] be a new array
2  for i = 0 to n − 1
3      make B[i] an empty list
4  for i = 1 to n
5      insert A[i] into list B[⌊n · A[i]⌋]
6  for i = 0 to n − 1
7      sort list B[i] with insertion sort
8  concatenate the lists B[0], B[1], . . . , B[n − 1] together in order
9  return the concatenated lists
```

Homework (due with Assignment 1)

- What is the running time of QUICK SORT when all elements of the array have the same value?
- An array A of size is $n>10$ is filled as follows:

  For each element $A[i]$, choose two random variables x_i and y_i uniformly and independently from $[0,1)$. Then set:

$$A[i] = \frac{\lfloor 10x_i \rfloor}{10} + \frac{y_i}{n}$$

  Modify BUCKET SORT so that it sorts the array A in $O(n)$ expected time.